

RAPPORT DE PROJET

MESURE DE LA POSITION D'UN ROBOT
MARIN DANS LE BASSIN D'ESSAI

Parcours SYSMER
2A Ecole d'Ingénieur
2025-2026



LABORATOIRE : COSMER

ENCADRANTS : CLAIRE DUNE,
LOÏCK DEGORE

EQUIPE :

COULONGEAT Théo,
BRULIN Romain,
DUBUC Julien,
CLAVERIE Timm

Nous souhaitons remercier nos professeurs pour leur accompagnement, Dre Claire Dune et Dr Loïck Degorre, pour leur temps, pour nous avoir transmis leurs connaissances et nous avoir guidés dans les domaines de la vision par ordinateur et de la robotique.

Résumé

Pour tester les robots sous-marins en bassin d'essai, il est indispensable de connaître précisément leur trajectoire. Actuellement, le laboratoire COSMER utilise Qualisys, un système de motion capture infrarouge. Bien qu'il offre une précision millimétrique, il présente des inconvénients : un coût très élevé, un équipement encombrant, un temps d'installation d'une demi-journée et l'obligation de fixer des marqueurs physiques sur le robot.

L'objectif de notre projet est de proposer une alternative précise, rapide à installer et moins coûteuse, en utilisant des caméras standards et sans rien modifier sur le robot. Pour y arriver, nous avons combiné l'intelligence artificielle YOLOv8 pour repérer le robot malgré les reflets et les déformations de l'eau, avec des mathématiques de triangulation pour calculer sa position 3D en temps réel.

Pour que ce travail soit directement exploitable par les chercheurs du laboratoire, nous avons développé un pipeline Python autonome, capable d'enregistrer et d'exporter les coordonnées 3D. Et, pour vérifier si notre système était vraiment fiable, nous avons comparé nos mesures avec celles de Qualisys lors de plusieurs tests pratiques (dans l'air, puis dans le bassin).

Mots-clés : Vision par ordinateur, ROV, BlueROV, YOLOv8, Triangulation 3D, chaîne de traitement Python, Suivi de trajectoire, étalonnage intrinsèque, étalonnage extrinsèque.

Table des matières

1	Introduction	3
1.1	Gestion de projet et organisation de l'équipe	4
1.1.1	Planification avec le diagramme de Gantt	4
1.1.2	Répartition du travail avec la Matrice Raci	4
1.1.3	Outils de communication	4
1.2	Analyse Fonctionnelle	4
1.2.1	Expression du besoin (Bête à cornes)	4
1.2.2	Interactions et contraintes (Diagramme Pieuvre)	5
1.3	État de l'art	6
1.3.1	Les approches géométriques classiques	6
1.3.2	L'apprentissage profond (Deep Learning)	7
1.4	Démarche expérimentale du projet	7
2	Vision par ordinateur	8
2.1	Méthodes de Vision Classiques (OpenCV)	8
2.1.1	Le filtre HSV	8
2.1.2	CamShift	8
2.1.3	L'algorithme ORB	8
2.1.4	Une méthode hybride (HSV + Tracker + ORB)	9
2.2	Méthode par Intelligence Artificielle	9
2.2.1	YOLOv8	9
2.2.2	SAM	10
2.2.3	CosyPose	10
2.2.4	Bilan	10
3	Modélisation Mathématique et 3D	11
3.1	Étalonnage du système multcaméras	11
3.1.1	Étalonnage Intrinsèque	11
3.1.2	Étalonnage Extrinsèque	12
3.2	Reconstruction Spatiale	14
3.2.1	Triangulation 3D	14
3.2.2	Redressement du repère	14
4	Architecture et Expérimentation	16
4.1	Organisation du Système	16
4.2	Tests dans l'Air	16
4.2.1	Protocole Expérimental	16

4.2.2	Analyse des Résultats - Test 1	21
4.2.3	Analyse des Résultats - Test 2	22
4.3	Tests en Bassin	23
4.3.1	Protocole et Adaptations	23
4.3.2	Analyse des Résultats - Test 1	25
4.3.3	Analyse des Résultats - Test 2	25
4.3.4	Bilan	27
4.4	Analyse de la précision du système	28
5	Conclusion	29
5.1	Respect du Cahier des Charges Fonctionnel	29
5.2	Améliorations	30
5.3	Ouverture	31
	Bibliographie	31
6	Annexe	33
6.1	GitHub	33
6.2	Gestion de projet et Analyse Fonctionnelle	33
6.3	Traitement d'images	36
6.3.1	Méthodes de Vision Classiques (OpenCV)	36
6.3.2	Filtrage colorimétrique (HSV)	36
6.3.3	Suivi par histogramme (CamShift)	37
6.3.4	Détection par caractéristiques (ORB)	37
6.3.5	Stratégie hybride (HSV/ORB)	38
6.4	Méthode par Intelligence Artificielle	40
6.4.1	YOLO v8	40
6.4.2	SAM	41
6.4.3	CosyPose	42
6.5	Modèles de positionnement 3D	44
6.5.1	Code étalonnage intrinsèque	44
6.5.2	Étalonnage extrinsèque	44
6.5.3	Triangulation 3D	46
6.5.4	Redressement repère	48
6.6	Manuel d'utilisation	50
6.6.1	Livrables — Dépôts GitHub	50
6.6.2	Prérequis et Matériel	51
6.6.3	Pipeline d'utilisation (Étape par étape)	51
6.7	Post traitement : Comparaison Qualisys	54
6.8	Analyse de la précision du système	55

1 Introduction

Pour valider le comportement d'un robot marin, il faut connaître sa position en temps réel dans le bassin d'essai. Le système actuel du laboratoire COSMER — Qualisys [10] — offre une précision millimétrique, mais son déploiement est contraignant : 5 caméras infrarouges, une calibration longue, des marqueurs physiques sur le robot et un coût élevé.

Ce projet propose une alternative basée sur des caméras standard. L'objectif est de suivre le BlueROV en temps réel, sans modifier le robot, via des algorithmes de vision par ordinateur structurés sous la forme d'un pipeline Python, qui seront intégrés plus tard dans l'environnement ROS2 [9] du laboratoire.

Le projet se divise en trois axes :

- **Détection** : comparer les méthodes classiques d'algorithmes OpenCV et d'intelligence artificielle comme YOLO en milieu aquatique.
- **Modélisation** : étalonnage des caméras et triangulation 3D.
- **Intégration** : livrer une architecture Python documentée et exploitable par les chercheurs du laboratoire COSMER.

La performance du système sera évaluée en comparant nos résultats avec ceux de Qualisys lors de tests en air et dans le bassin.

Le système est organisé de telle manière que l'évaluation de la trajectoire soit possible en temps réel ou en post-traitement en liant plusieurs étapes d'étalonnage, d'enregistrement de flux vidéo et de reconstruction 3D. Pour éviter que l'utilisateur interagisse avec le code, nous avons rendu le système autonome et créé une IHM fonctionnelle.

Le rendu est disponible sur GitHub à la page suivante : <https://github.com/Projet-BlueROV-Seatech>. Il est public et accompagné d'un ReadMe pour proposer une alternative OpenSource à Qualisys pour les différents laboratoires de recherche et autres utilisateurs.

1.1 Gestion de projet et organisation de l'équipe

Pour mener à bien ce projet, nous avons dû mettre en place une organisation rigoureuse. L'objectif : coordonner le travail de quatre étudiants sur plusieurs semaines, tout en gérant l'accès au matériel (caméras, robot) et aux infrastructures (bassin d'essai).

1.1.1 Planification avec le diagramme de Gantt

Le diagramme de Gantt a été notre fil rouge, sa figure est en annexe 6.1. Il nous a permis de :

- Fixer nos deadlines pour chaque grande étape.
- Classer les tâches par ordre de priorité.
- Repérer les dépendances. Par exemple, la calibration des caméras devait être terminée avant de commencer la triangulation 3D.

1.1.2 Répartition du travail avec la Matrice Raci

Pour organiser le travail, nous utilisons un tableau de bord et faisons un point rapide à chaque début et fin de séance. La répartition des rôles est résumée par la matrice RACI sur la figure 6.2 disponible en annexe.

Cette méthode nous a évité de travailler chacun dans notre coin, elle a permis de garder nos scripts compatibles et de répartir le travail intelligemment.

1.1.3 Outils de communication

- Google Drive partagé avec nos deux enseignants avec rangement intelligent pour éviter d'écraser le code d'un de nos camarades.
- Messagerie pour discuter avec les membres de l'équipe et mails ou présentiel pour échanger, demander le matériel et faire les tests.

1.2 Analyse Fonctionnelle

L'analyse fonctionnelle a pour but de cadrer notre développement et de traduire les attentes du laboratoire COSMER en fonctions techniques et en contraintes que notre système devra respecter.

1.2.1 Expression du besoin (Bête à cornes)

Afin de déterminer le besoin auquel répond notre projet, nous avons modélisé la situation avec un diagramme Bête à cornes (voir figure 1.1)

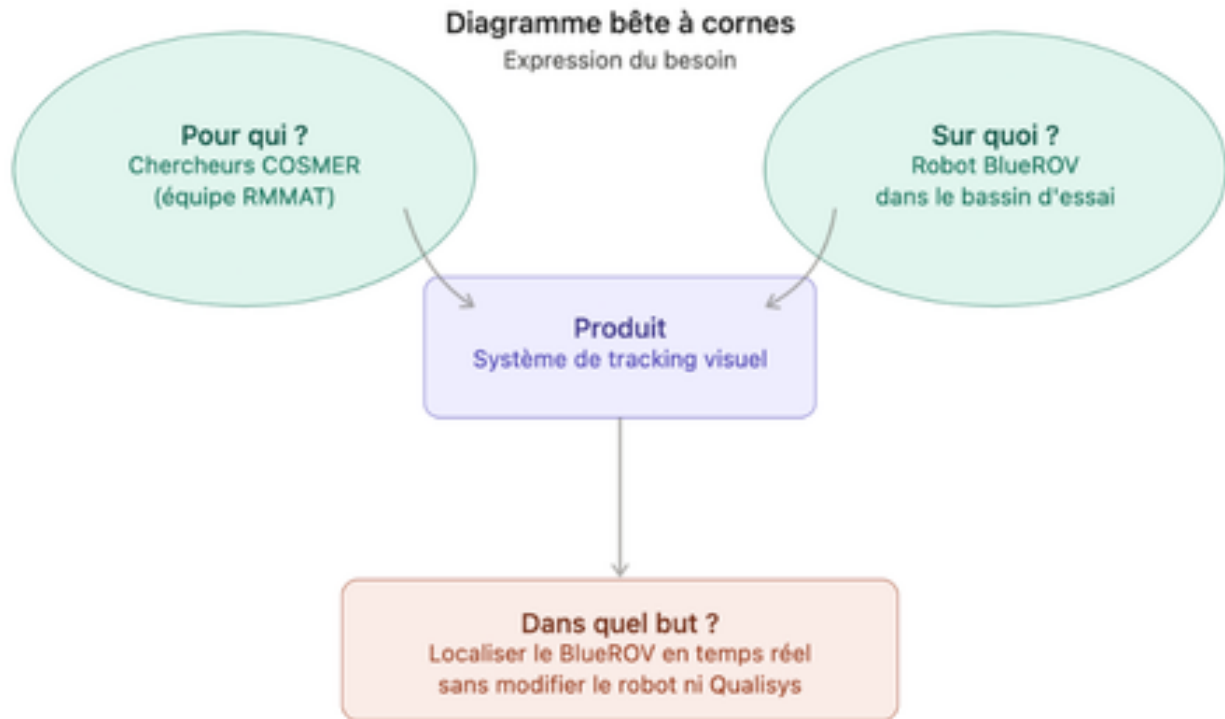


FIGURE 1.1 – Diagramme Bête à cornes

L'outil s'adresse aux chercheurs du COSMER (équipe RMMAT). Le système doit détecter le BlueROV et fournir sa position (3D) dans le bassin via des caméras standard. Le livrable est un pipeline logiciel Python qui doit offrir une solution de localisation visuelle plus souple et plus rapide à déployer que le système Qualisys actuel.

1.2.2 Interactions et contraintes (Diagramme Pieuvre)

Pour définir le cahier des charges, nous avons identifié les interactions de notre système avec son environnement via un diagramme Pieuvre (voir figure 6.3).

Ce diagramme permet de définir le Cahier des Charges Fonctionnel (CDCF) synthétisé dans le tableau 1.1.

Réf.	Fonction	Critère d'appréciation	Niveau attendu
FP1	Fournir la position 3D du BlueROV à l'utilisateur	Erreur de position vs Qualisys	< 7 cm
FC1	Fournir une architecture Python exploitable et documentée	Livrables fournis	Codes et README sur GitHub
FC2	Fonctionner avec des caméras standards	Compatibilité matérielle	Caméras USB grand public
FC3	Résister aux perturbations visuelles	Taux de détection correct	$\text{IoU} > 0.5$
FC4	Traiter le flux vidéo en temps réel	Fréquence de traitement	> 15 fps sur CPU
FC5	Ne pas nécessiter de modification du robot	Marqueurs physiques ajoutés	Zéro modification

TABLE 1.1 – Cahier des charges fonctionnel (CDCF)

Ce tableau met en évidence une Fonction Principale (FP1) et cinq Fonctions Contraintes. FP1 exprime le besoin : fournir la position 3D du BlueROV à l'utilisateur avec une marge d'erreur acceptable. Les fonctions contraintes délimitent le développement : le système doit prendre la forme d'une architecture Python (FC1), fonctionner de manière non-intrusive avec des caméras standards (FC2, FC5), résister aux perturbations optiques du milieu aquatique (FC3) et maintenir une fréquence de traitement garantissant le temps réel (FC4).

1.3 État de l'art

Suivre un ROV dans un environnement contrôlé comme un bassin d'essai est largement documenté. La difficulté principale vient des perturbations aquatiques : réfraction, reflets de surface, uniformité des couleurs. Dans la littérature, deux familles de méthodes se distinguent pour résoudre ce problème.

1.3.1 Les approches géométriques classiques

Historiquement, le positionnement visuel s'appuie sur l'extraction de caractéristiques géométriques (algorithmes ORB ou SIFT) couplée à un algorithme PnP pour estimer la pose spatiale de l'objet. L'étalonnage multicaméras réalisé par détection de marqueurs (ArUco ou AprilTags) est formalisé par Zhang [2] et implémenté dans la bibliothèque OpenCV [1], cet étalonnage constitue généralement la vérité terrain mathématique en recherche.

Mais, ces méthodes nécessitent des cibles texturées ou des marqueurs physiques car la littérature parle de leur sensibilité au bruit visuel généré par les mouvements de l'eau, ce qui contredit notre

contrainte d'usage FC5 (ne pas modifier le robot).

1.3.2 L'apprentissage profond (Deep Learning)

Face à ces limites, la recherche s'est massivement tournée vers les réseaux de neurones. Les architectures de détection par boîtes englobantes (bounding boxes), comme YOLO [4, 5] constituent aujourd'hui le standard industriel pour la détection temps réel grâce à leur robustesse aux variations d'éclairage.

D'autres approches plus avancées existent, comme la segmentation au pixel près avec SAM [6] ou l'estimation de pose 6D avec CosyPose [7]. Bien qu'elles offrent une information supérieure (détourage exact, inclinaison 3D), elles nécessitent des ressources de calcul importantes, ce qui les rend souvent incompatibles avec le temps réel sur des ordinateurs portables standards.

1.4 Démarche expérimentale du projet

En partant de cet état de l'art et des contraintes fixées par notre Cahier des Charges Fonctionnel, il n'existe pas de solution évidente pour notre environnement.

Pour répondre au besoin du laboratoire COSMER, nous avons structuré notre démarche scientifique en trois grandes étapes :

1. **Étude comparative pour la détection 2D (Chapitre 2) :** Avant de choisir une solution, nous allons expérimenter et comparer les algorithmes classiques de traitement d'images (OpenCV) avec les algorithmes d'intelligence artificielle (YOLO, SAM, CosyPose) afin d'identifier la méthode la plus robuste face au milieu aquatique et qui fonctionne en temps réel.
2. **Reconstruction spatiale (Chapitre 3) :** Une fois la cible détectée en 2D sur l'image, nous modéliserons mathématiquement un système de triangulation multicaméras, nécessitant des étapes d'étalonnage pour extraire les coordonnées 3D (X, Y, Z) du robot en mètres. Cette étape de reconstruction à partir de données segmentées est une méthode classique en vision par ordinateur [11].
3. **Validation et Intégration (Chapitres 4 et 5) :** L'ensemble de notre méthode sera évalué expérimentalement face à la vérité terrain Qualisys [10], avant d'être mis à disposition sous la forme de codes Python.

2 Vision par ordinateur

Il existe deux méthodes de vision par ordinateur pour traiter les frames d'un flux vidéo, le traitement d'images par algorithme et par intelligence artificielle. Nous allons tester et comparer ces méthodes pour déterminer laquelle ou lesquelles sont les plus adaptées à la détection et au suivi de ROV.

2.1 Méthodes de Vision Classiques (OpenCV)

Notre premier objectif a été de repérer le robot avec la librairie OpenCV [1]. Nous avons testé quatre méthodes classiques pour savoir si elles étaient fiables et rapides.

2.1.1 Le filtre HSV

Ce filtre permet d'isoler la masse sombre du robot en se basant sur sa couleur.

2.1.2 CamShift

C'est un algorithme qui suit dynamiquement le déplacement des couleurs du robot sur la vidéo. Sous l'eau, les reflets et les ombres de l'eau font rapidement échouer cet algorithme. (Voir figure 2.1)

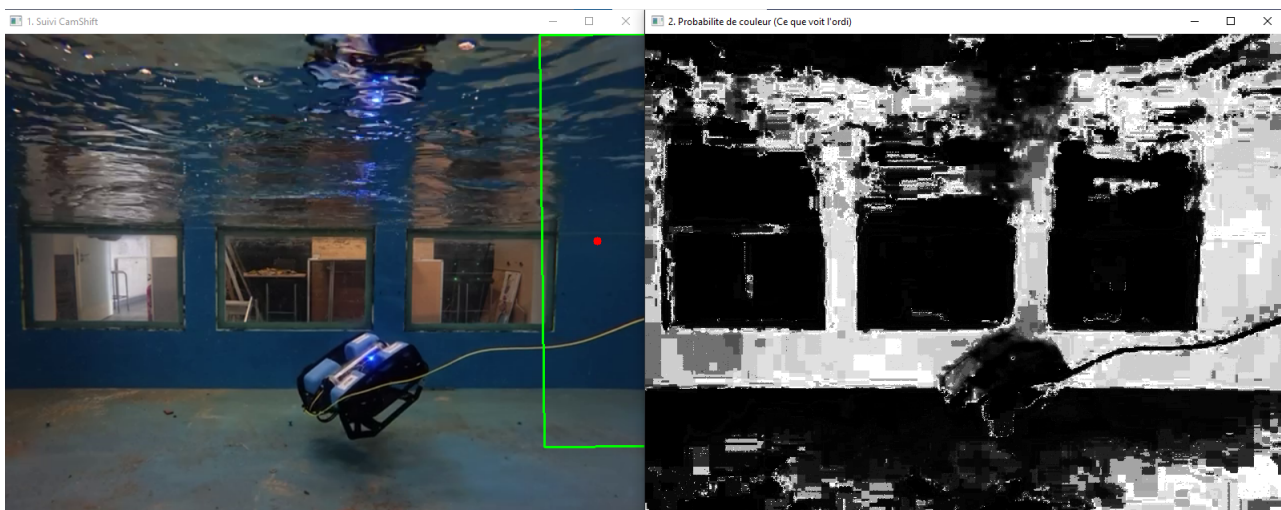


FIGURE 2.1 – Perte du ROV avec CamShift

2.1.3 L'algorithme ORB

Cet algorithme tente de repérer les angles et les points d'intérêt du robot à partir de son modèle CAO 3D.

2.1.4 Une méthode hybride (HSV + Tracker + ORB)

Nous avons tenté de fusionner le filtre HSV et l'algorithme ORB afin d'avoir un meilleur tracking du robot.

Bilan : pourquoi passer à l'IA ?

Malgré une méthode hybride assez aboutie, l'environnement sous-marin a mis en échec ces méthodes pour trois raisons :

1. L'eau agit comme un filtre qui diminue fortement les contrastes et change les couleurs avec la profondeur.
2. Les reflets à la surface et les ombres au fond de l'eau créent des parasites qui trompent nos filtres de couleur.
3. La coque du BlueROV est lisse, l'algorithme ORB ne trouve donc pas d'angles nets pour s'accrocher.

Ainsi, le système finissait toujours par perdre la trace du robot en glissant sur des reflets (voir figure 2.1). Cet échec de la vision classique justifie notre passage à l'Intelligence Artificielle (YOLO). Contrairement à de simples filtres, le Deep Learning est capable d'apprendre la forme globale du robot, même quand les détails et les couleurs sont déformés par l'eau.

(L'analyse de ces tests, les résultats et nos codes OpenCV sont disponibles en Annexe 6.3.1).

2.2 Méthode par Intelligence Artificielle

Notre objectif est de positionner dans l'image le centre du BlueROV en utilisant des méthodes par IA fiables, dont la qualité peut être mesurée et qui fonctionnent en temps réel.

2.2.1 YOLOv8

L'entraînement de YOLOv8 sur le BlueROV a nécessité la création d'un dataset : extraction d'une frame toutes les 10 secondes, annotation manuelle avec FiftyOne, séparation train/val. L'entraînement dure environ 10 minutes pour 1 000 images sur GPU. Le modèle est sensible aux changements de luminosité entre sessions, un réentraînement est donc nécessaire à chaque nouvelle condition. *(Le détail de l'entraînement est disponible en Annexe 6.4)*

En détection, YOLO fournit une boîte englobante (*bounding box*) dont on extrait le centre en pixels (position 2D). La limite principale est que le modèle peut confondre le robot avec son reflet en surface s'il n'est pas bien entraîné. La qualité de détection est mesurée par l'IoU (Intersection of Union), ici les deux boîtes se superposent quasiment parfaitement avec un IoU proche de 1, (voir figure 2.2) ce qui traduit une détection très performante.



FIGURE 2.2 – Contrôle qualité YOLO

2.2.2 SAM

Le modèle de segmentation SAM (*Segment Anything Model*) a été testé pour détourner le robot au pixel près de manière autonome. Mais, son architecture s’est révélée beaucoup trop lourde pour nos PC, rendant impossible le traitement vidéo en temps réel. (*Comparatif des temps de calcul en Annexe 6.4.2*)

2.2.3 CosyPose

Ce modèle permet d’estimer la pose 6D complète du robot (position spatiale et angles d’inclinaison). Cependant, son entraînement exige la génération d’un immense jeu de données (via *BlenderProc*) qui peine à simuler les perturbations aquatiques. (*Détails des tests en Annexe 6.4.3*)

2.2.4 Bilan

Après avoir vérifié que le besoin du laboratoire se limitait au suivi de la trajectoire 3D (x, y, z) , nous avons abandonné les méthodes trop lourdes (SAM et CosyPose) à mettre en œuvre au profit de la légèreté de YOLO.

3 Modélisation Mathématique et 3D

Pour évaluer la trajectoire du robot, il faut asservir le système multicaméras et estimer la position 3D du ROV de manière séquentielle.

3.1 Étalonnage du système multicaméras

Des imprécisions propres aux milieux et aux caméras détériorent la perception et l'estimation de la position 3D du ROV. Les corrections à effectuer sont propres aux expérimentations et doivent être réalisées en amont d'un suivi de trajectoire.

3.1.1 Étalonnage Intrinsèque

Méthode

Afin de calculer la position 3D du BlueROV, nous avons décidé d'utiliser des caméras. Cependant, leurs objectifs déforment naturellement les images comme avec la distorsion radiale. L'étalonnage intrinsèque a pour but de modéliser mathématiquement chaque lentille des caméras pour corriger ces déformations [2].

Pour ce faire, nous avons filmé une mire de calibration, composée de marqueurs AprilTag qui permettent une détection beaucoup plus robuste que les damiers classiques [3].

L'algorithme détecte les coins des marqueurs pour faire le lien entre leurs dimensions réelles et leur position sur l'image. Le calcul des matrices de la caméra est plus précis si on filme la mire sous des angles différents.

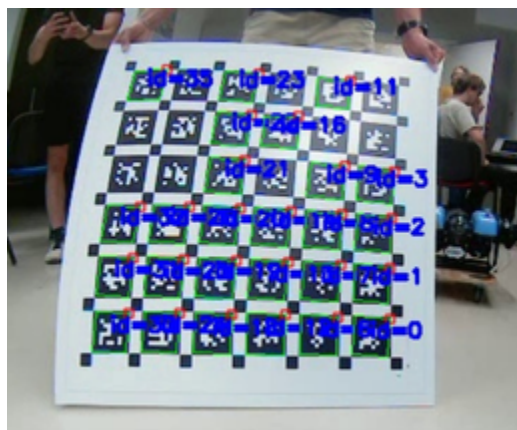


FIGURE 3.1 – Détection des marqueurs pour l'étalonnage intrinsèque

Cependant, durant ces calibrations, nous avons rencontré différents problèmes. Tout d'abord, si on bougeait la mire trop vite, les coins devenaient illisibles et l'ordinateur se trompait. Il a donc

fallu réaliser des mouvements lents pour pouvoir récolter suffisamment d'images nettes et valider l'étalonnage.

Code

Nous avons développé un script basé sur le module `cv2.ArUco` d'OpenCV. Il localise les coins des marqueurs avec une précision sub-pixel, puis calcule mathématiquement la matrice intrinsèque de la caméra (K) et ses coefficients de distorsion (D). (*La logique du code est disponible en Annexe 6.5.1*).

Test de qualité

Pour s'assurer que notre étalonnage intrinsèque était correct, nous nous sommes basés sur l'erreur RMS (*Root Mean Square*) qui représente l'écart moyen (en pixels) entre l'endroit où notre modèle pense que le point devrait être et celui où il se trouve réellement sur la photo. Nous avons fait un algorithme pour voir la correction appliquée à l'image (Voir figure 3.2).

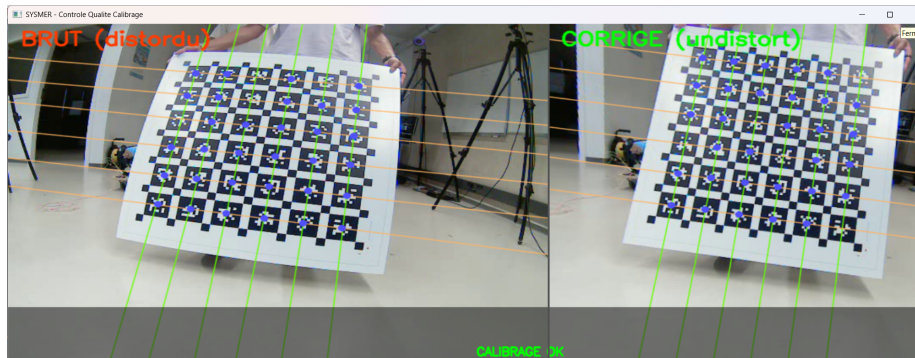


FIGURE 3.2 – Image Avant VS Après Étalonnage Intrinsèque

L'erreur est acceptable à partir de 1.5 de RMS et doit être le plus proche de 1 pixel. Si l'erreur RMS dépasse 1.5 pixels, il faut recommencer. Cela nous indique immédiatement que la vidéo manquait de diversité angulaire ou contenait trop de flou, nous forçant ainsi à recommencer proprement. À l'inverse, si le score est sous ce seuil, les matrices sont validées pour la suite du projet.

3.1.2 Étalonnage Extrinsèque

Méthode

Les positions XY d'un objet sur les plans images de plusieurs caméras permettent d'obtenir la position 3D de l'objet. La profondeur Z de l'objet s'obtient en calculant la position du point du croisement des droites, partant du point focal vers le point image où l'objet est détecté, sur chacune des caméras. Pour que ces lignes se croisent, il faut connaître la position 6D des caméras, c'est à dire, leurs angles et leurs coordonnées dans l'espace 3D. Des imprécisions sur la position rendraient la détection 3D de l'objet impossible car les droites ne se croiseraient peut-être jamais.

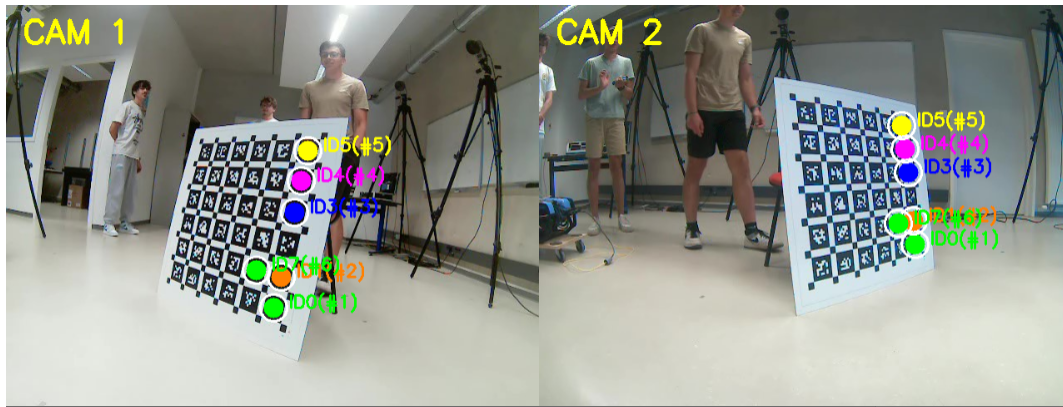


FIGURE 3.3 – Reconnaissance d'ArUcos pour modèle PNP

L'étalonnage extrinsèque permet à partir d'au moins 5 points différenciables d'estimer des matrices de rotation et de translation entre 2 caméras. Ainsi, en choisissant l'une des caméras comme origine du repère, la position 6D ne devient plus une source d'erreur sur l'estimation de la position 3D du ROV (par rapport au repère caméra).

Notre modèle mathématique d'étalonnage extrinsèque est principalement basé sur la méthode de Nelder-Mead qui correspond à la minimisation d'une fonction de calcul de coût. Les relations mathématiques et informatiques du modèle sont expliquées en Annexe 6.5.2.

Test de qualité

Le système de capture Qualisys permet de mesurer la qualité de la translation entre les positions des deux caméras. Sur notre modèle python, la calibration peut se faire avec au moins 6 points mais il en faut au moins 20 pour être précis.

Dans le bassin, nous avons comparé la distance entre les deux caméras trouvée par notre algorithme et celle mesurée physiquement avec un mètre. (Voir figure 3.4)

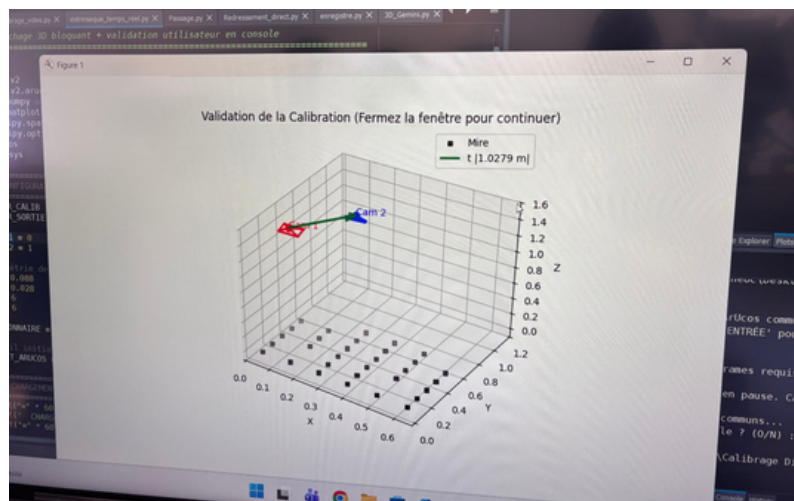


FIGURE 3.4 – Contrôle Qualité Extrinsèque Bassin

Cela nous a permis de conclure que notre algorithme d'étalonnage était fiable et précis.

3.2 Reconstruction Spatiale

Une fois les images rectifiées, les matrices de passage entre les caméras obtenues et les points des plans images récupérés, nous déterminons la position 3D du ROV par rapport au centre optique de la caméra. Celle-ci étant inclinée, il faut redresser le repère pour obtenir l'altitude réelle des points 3D par rapport au sol.

3.2.1 Triangulation 3D

Connaissant les matrices de projection P_1 et P_2 de l'étalonnage, la fonction `cv2.triangulatePoints` trace un rayon optique depuis le centre focal de chaque caméra vers le point détecté par YOLO. (Voir figure 3.5)

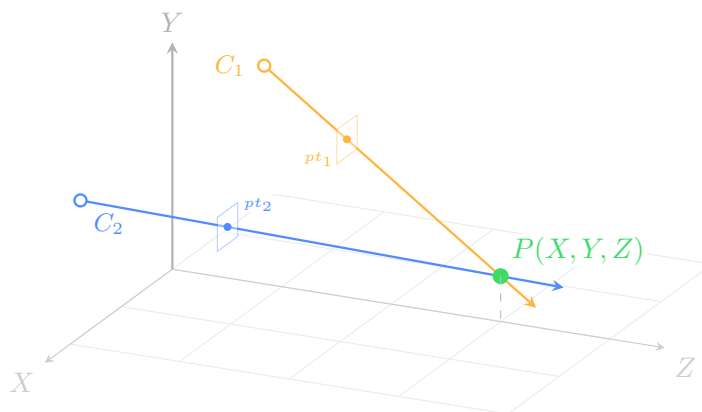


FIGURE 3.5 – Triangulation 3D

L'intersection spatiale de ces deux rayons donne la position (X, Y, Z) du robot en mètres. En pratique, les erreurs numériques et le volume physique du robot font que ces droites ne se croisent jamais parfaitement dans l'espace. L'algorithme utilise donc la méthode des moindres carrés ($\min \|AX\|^2$) pour estimer le point de croisement le plus probable.

(Le développement matriciel complet est détaillé en Annexe 6.5.3).

3.2.2 Redressement du repère

Nos caméras étant inclinées vers le bassin, les coordonnées 3D calculées par la triangulation le sont aussi. L'objectif du redressement est de remettre ces coordonnées à plat, dans un repère aligné avec le sol et la gravité.

L'algorithme détecte la mire posée à plat et utilise des outils mathématiques (méthode *solvePnP*, produits vectoriels) pour calculer l'inclinaison exacte de la caméra, recrée les axes (X, Y, Z) parfaitement droits et crée une matrice de redressement. (Voir figure 3.6)

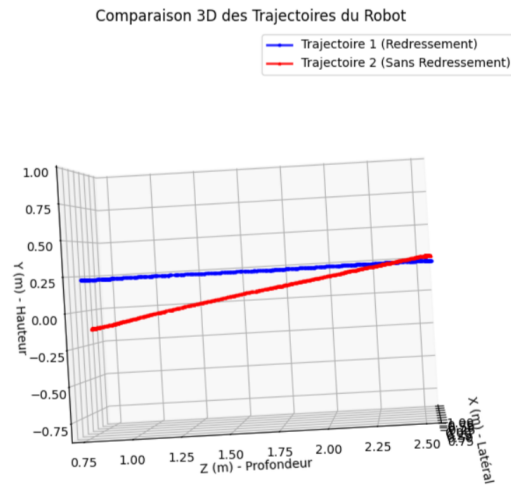


FIGURE 3.6 – Trajectoire du ROV Avant et Après Redressement du repère

En temps réel, elle corrige la position du robot pour nous donner sa vraie profondeur et sa vraie position dans le repère du bassin.

(Les détails de ces calculs sont disponibles en Annexe 6.5.4).

Test qualité

Nous avons comparé les résultats de hauteur, donnés lors des tests après redressement, avec la mesure de la hauteur du centre du ROV dans la réalité. Nous en avons conclu que notre méthode de redressement était correcte.

4 Architecture et Expérimentation

Ce chapitre aborde de manière précise la manière dont est organisée la chaîne d'action (Pipeline Python). Différentes expérimentations ont été menées à travers une série de tests en air et en bassin. pour tester et analyser notre système.

4.1 Organisation du Système

Notre système repose sur 5 scripts exécutés dans l'ordre.

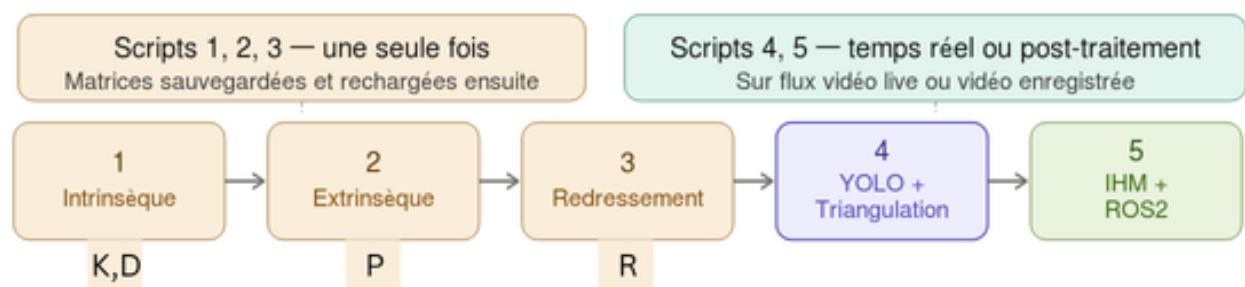


FIGURE 4.1 – Pipeline du système

Les scripts 1, 2 et 3 (étalonnages + redressement) sont exécutés une seule fois et produisent des matrices sauvegardées sur l'ordinateur. Les scripts 4 et 5 (YOLO, triangulation, IHM) fonctionnent en temps réel sur flux vidéo live ou en post-traitement sur vidéo enregistrée. Les résultats sont validés par la comparaison avec Qualisys.

(Le ReadMe avec tous les détails est disponible en Annexe 6.6).

(La méthode de comparaison à Qualisys est aussi disponible en annexe 6.7).

4.2 Tests dans l'Air

Avant de tester nos codes et notre installation dans le bassin, il était nécessaire de savoir si tout était fonctionnel dans un milieu contrôlé afin de valider la précision de notre suivi de trajectoire par rapport au système Qualisys (référence).

4.2.1 Protocole Expérimental

Matériel

Pour réaliser ce test, nous avons utilisé le matériel suivant :

Système visuel (Notre solution) :

- Deux caméras USB standard grand public (DeepWater - exploreHD (400m)).

- Un PC portable pour l'acquisition et le traitement temps réel (YOLO, Triangulation).
- Le robot BlueROV.
- Mire d'étalonnage (85 cm) composée d'une grille 6×6 de marqueurs *AprilTag* (DICT_APRILTAG_36h11). Chaque marqueur mesure 8,8 cm avec un espacement de 2,8 cm. Cet outil est indispensable pour l'étalonnage intrinsèque et extrinsèque. (Voir figure 4.2)

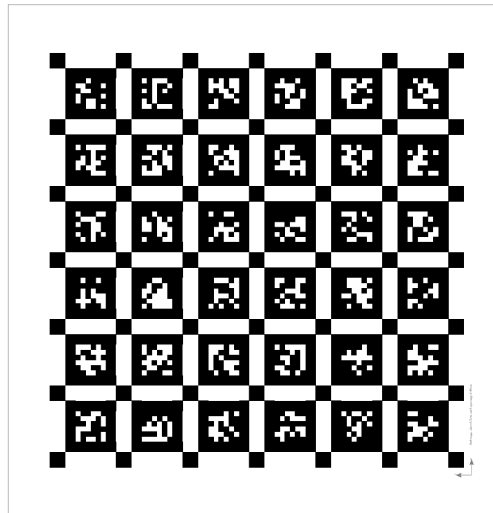


FIGURE 4.2 – Mire d'étalonnage

Système Qualisys pour le contrôle qualité :

- 5 caméras Qualisys (caméras infrarouges de *motion capture*) installées autour de la zone de test.
- Des marqueurs passifs (petites boules réfléchissantes) fixés sur le châssis du BlueROV et sur nos deux caméras USB, qui permettent de connaître la position exacte (millimétrique) du robot et de nos caméras.

Autre :

- Une planche à roulette pour déplacer le BlueROV.
- Des cordes attachées au chariot pour manipuler la cible sans entrer dans le champ de vision des caméras.
- Du scotch collé pour tracer un plan au sol pour vérifier visuellement la trajectoire mesurée.

Installation

Nous avons installé des caméras de manière orthogonale sur des chaises et légèrement orientées vers le sol, approximativement à la même hauteur, que nous avons ensuite branchées à un PC.

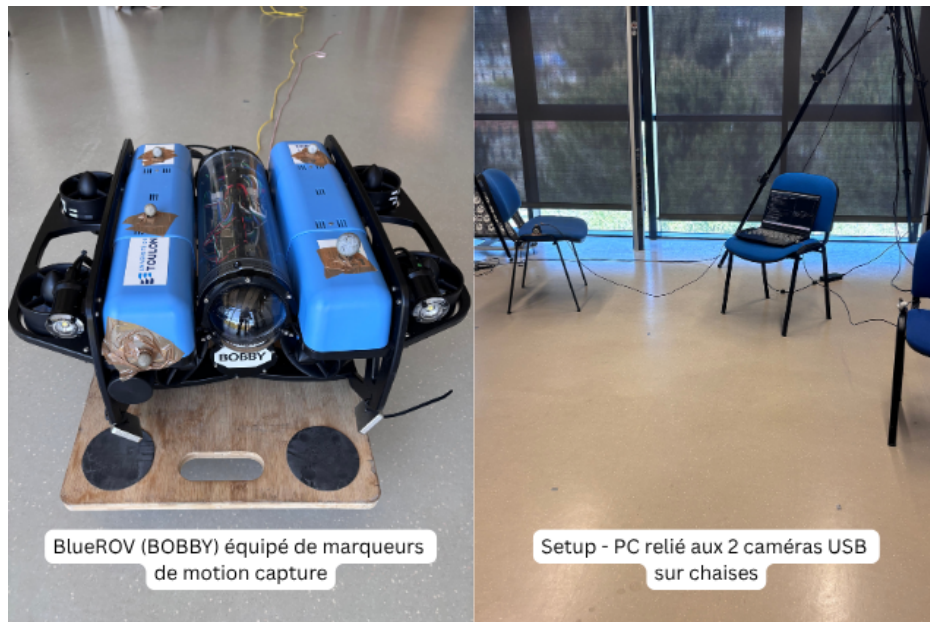


FIGURE 4.3 – Installation expérimentale pour les tests dans l'air

Mise en place du système Qualisys

Afin de pouvoir évaluer précisément notre propre code, nous devons d'abord configurer le système Qualisys pour qu'il enregistre la trajectoire parfaite du robot et la position de nos caméras. Ce protocole s'est déroulé en quatre étapes :

1. **Étalonnage** : Nous avons placé une équerre de calibration au sol pour définir l'origine et les axes (X, Y, Z) du repère monde. Ensuite, nous avons agité une baguette dynamique munie de marqueurs (le *T-wand*) dans tout le volume de l'expérience. Cela permet aux 5 caméras infrarouges de cartographier l'espace 3D avec une précision millimétrique.
2. **Zone de travail** : Pour éviter de perdre le signal du robot en cours de test, nous avons fait rouler le chariot dans la pièce pour délimiter les limites de couverture des caméras Qualisys en collant du scotch au sol.
3. **Positionnement des capteurs** : Des marqueurs passifs ont été fixés directement sur nos deux caméras USB pour enregistrer leurs coordonnées exactes dans l'espace, ce qui nous a permis de vérifier plus tard notre étalonnage extrinsèque.
4. **Acquisition** : Lors du déplacement du robot, Qualisys suit individuellement chaque petite boule réfléchissante. En post-traitement sur le logiciel, nous avons sélectionné le groupe de marqueurs collés sur le robot pour définir un corps rigide (*Rigid Body*) (Voir figure 4.4). Cela indique au logiciel que ces points forment un seul et même objet géométrique.

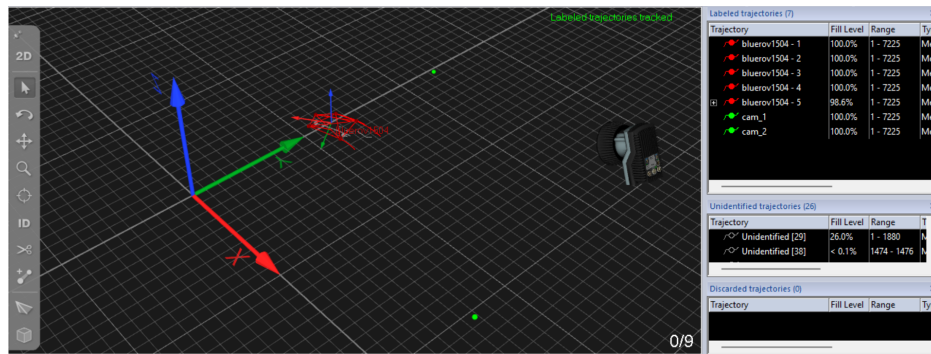


FIGURE 4.4 – Rigid Body du ROV et 2 caméras sur Logiciel Qualisys

- Export** : La trajectoire finale de ce corps rigide a ensuite été exportée au format `.tsv` pour être comparée à celle de notre algorithme.

Étalonnage Intrinsèque

Il est nécessaire d'effectuer d'abord l'étalonnage intrinsèque. Pour cela nous avons déplacé lentement la mire devant chaque caméra individuellement, dans plusieurs orientations différentes et à différentes positions de sorte à couvrir tout le champ de vision de la caméra car c'est là que le phénomène de distorsion est le plus important.

Il est nécessaire que le code d'étalonnage intrinsèque ait au moins 15 photos différentes avec 10 ArUco ou plus de détectés pour avoir un bon RMS proche de 1. Nous avons eu des RMS de 1,2342 et de 1,147 pour les caméras ce qui confirme la qualité de notre étalonnage.

Étalonnage Extrinsèque

Nous avons ensuite effectué l'étalonnage extrinsèque. Pour ce faire, il faut placer la mire pour qu'elle soit vue par les 2 caméras en même temps et que les caméras détectent au moins 20 ArUco en commun sur la mire. Le nombre d'ArUco détecté fait varier la précision du modèle. Il est possible de changer le nombre d'ArUco à détecter selon les besoins.

Nous avons vérifié et confirmé la qualité de notre algorithme extrinsèque en comparant la distance entre nos deux caméras donnée par notre algorithme et celle mesurée physiquement.

Redressement du repère

Suite à cela, nous devons redresser le repère de la caméra qui est utilisée comme origine. En effet, puisqu'elle est orientée vers le sol, son repère l'est aussi. Afin de le redresser, il faut donc placer la mire au sol devant la caméra servant d'origine et qu'elle détecte au moins 10 ArUco.

Suivi de trajectoire

Enfin, il est possible de passer au suivi de trajectoire 3D. Pour cela, il faut déplacer le BlueROV afin qu'il soit visible par les différentes caméras en même temps. Nous avons utilisé le chariot à roulettes tiré par des cordes pour simuler le déplacement du robot. Le dispositif Qualisys, visible sur la figure 4.5, enregistre simultanément la trajectoire exacte. L'IA YOLO permettra, dans

cette situation, d'obtenir le barycentre du robot et ensuite avec nos codes de nous retourner sa position dans l'espace.

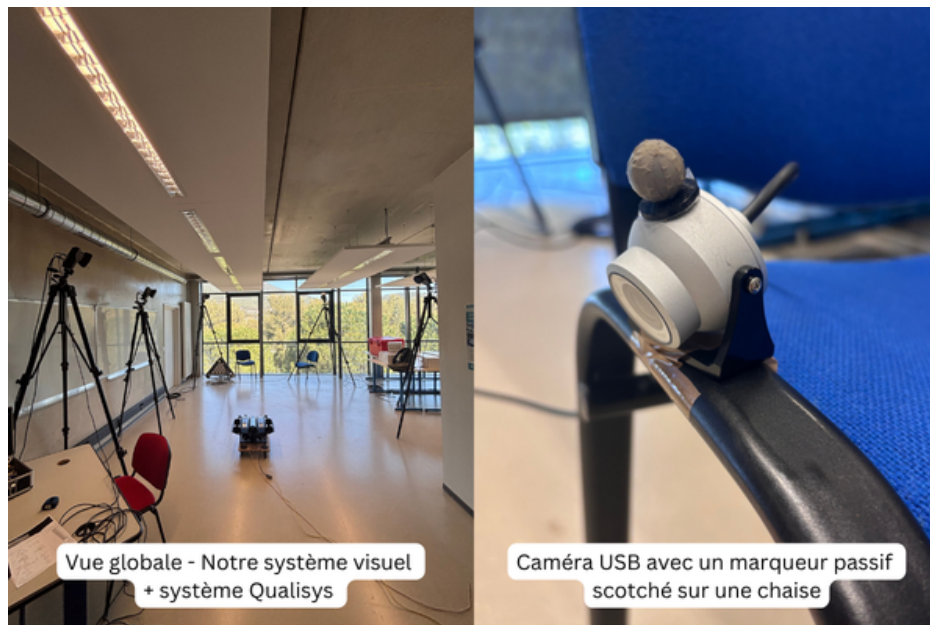


FIGURE 4.5 – Dispositif de validation Qualisys

4.2.2 Analyse des Résultats - Test 1

Le robot, posé sur un chariot à roulettes, a été tiré dans l'air par une corde. Les positions du BlueROV à étudier sont celles mesurées dans les zones de vision communes aux systèmes YOLO et Qualisys.

Comparaison avec le système Qualisys

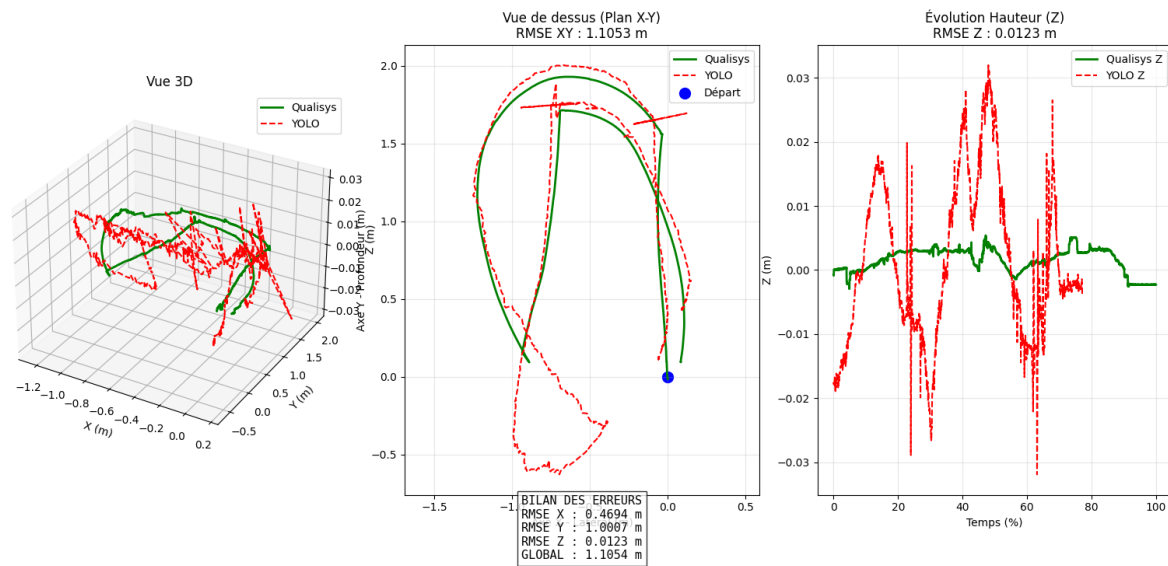


FIGURE 4.6 – Résultats Test 1 : Divergence des trajectoires (Sortie de zone)

Comme le montre la figure 4.6, ce premier essai a montré que le suivi sur le plan horizontal (X,Y) est erroné à plusieurs moments. En analysant ce qui s'était passé, nous avons identifié trois problèmes :

1. **Sortie de la zone Qualisys** : La majeure partie de l'erreur vient du fait que nous avons tiré le chariot en dehors de la vision des caméras Qualisys.
2. **L'angle des caméras** : En post-traitement, nous nous sommes rendu compte que l'algorithme pour étalonnage extrinsèque n'arrive pas à calculer correctement la matrice de passage lorsque l'angle entre deux caméras dépasse les 90 degrés.
3. **Les obstructions** : S'il y a un obstacle, la boîte de détection ne se fait plus sur tout le BlueROV, le centre de la boîte est alors déplacé ce qui donne un effet de déplacement perpendiculaire à la trajectoire et parallèle à la caméra. Nous observons ce cas deux fois ici.

Le changement de protocole :

Il ne faut plus enregistrer le test en une seule fois et espérer que le calcul fonctionne en post traitement. Nous adoptons donc une méthode étape par étape : on réalise l'étalonnage intrinsèque, on vérifie que le score RMS est bon. Ensuite, on fait l'étalonnage extrinsèque avec un angle aigu entre deux caméras, on vérifie que la distance calculée est cohérente. Une fois ces deux étapes validées, on passe à la triangulation 3D.

4.2.3 Analyse des Résultats - Test 2

Pour ce deuxième test, nous avons appliqué le nouveau protocole. L'angle entre les caméras est inférieur à 90° et la zone de travail de Qualisys est délimitée avec du scotch pour ne pas en sortir.

Comparaison avec le système Qualisys

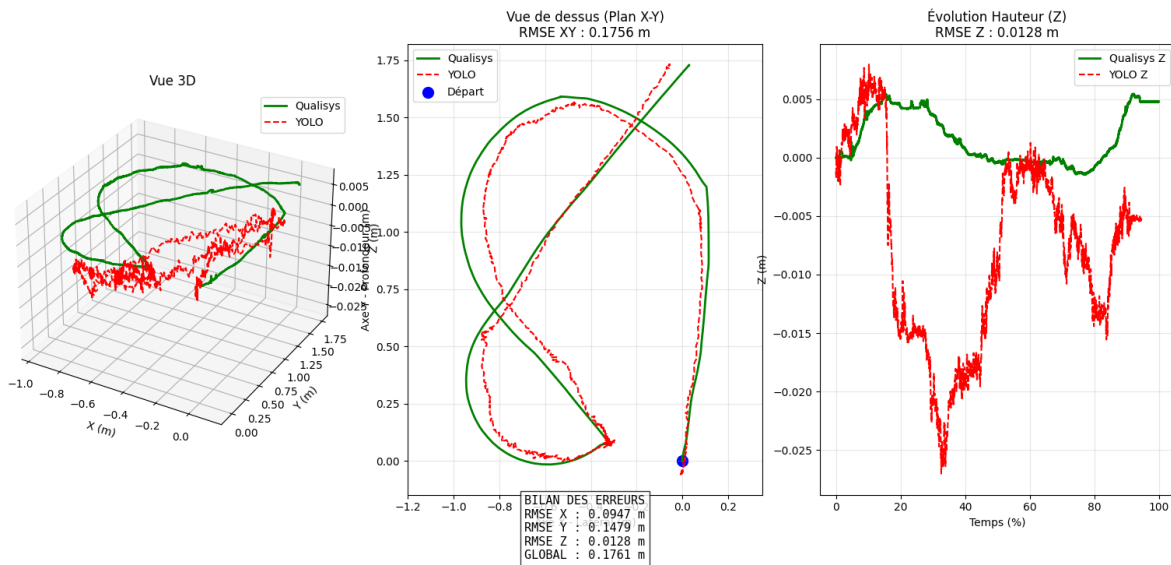


FIGURE 4.7 – Résultats du Test 2 : Après correction du protocole

Ce second test montre une amélioration comme l'indique la figure 4.7. La trajectoire estimée par notre système (en rouge) suit assez fidèlement la vérité terrain de Qualisys (en vert).

Cependant, on observe encore un léger décalage constant et du bruit (les petits sauts sur la courbe rouge) qui ne sont plus des erreurs de protocole, mais des erreurs cumulées liées à nos choix technologiques :

1. **Différence de cible :** Qualisys calcule la position du robot à partir des marqueurs collés sur la coque. De notre côté, on calcule le centre de la box trouvée par YOLO. Ce centre visuel change un peu selon l'angle de vue de la caméra. On ne traque donc pas exactement le même point physique, ce qui crée ce décalage constant.
2. **Bruit de YOLO :** L'IA YOLO n'est pas stable au pixel près. D'une image à l'autre, la boîte englobante peut changer légèrement de taille, ce qui impacte le calcul du barycentre et ce qui donne cet aspect en zigzag (sauts des pixels) par rapport à la courbe lisse de Qualisys.

Malgré ces erreurs cumulées que l'on ne peut pas totalement effacer, ce test valide notre travail. Avec de simples caméras standard, nous avons réussi à reconstruire la trajectoire 3D du robot de manière assez fiable.

4.3 Tests en Bassin

Après avoir validé la logique mathématique de notre code dans l'air, nous avons testé notre système dans son environnement final : le bassin d'essai du laboratoire COSMER. L'objectif était de vérifier si nos algorithmes de triangulation résistaient aux fortes perturbations optiques de l'eau.

4.3.1 Protocole et Adaptations

Le protocole général reste identique à celui réalisé dans l'air. Cependant, le passage sous l'eau a nécessité des adaptations matérielles :

- **Déploiement Qualisys** : Installation de 5 caméras Qualisys sous marine avec des supports (barres profilées ITEM) (très long et complexe), étalonnage du système Qualisys avec une équerre de calibration sous marine et agitation d'une baguette dynamique avec des marqueurs pour identifier les caméras et les étalonner. (Voir figure 4.8)

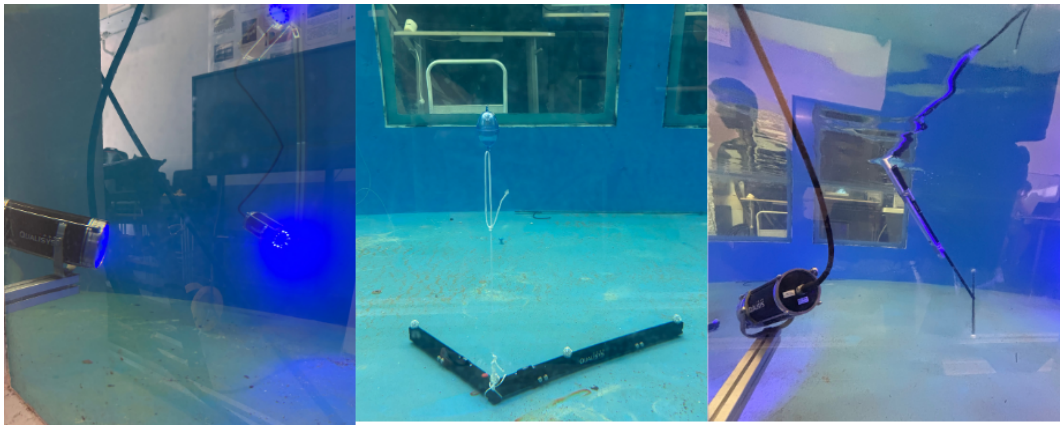


FIGURE 4.8 – Installation de Qualisys dans le bassin

- **Modification du ROV** : ROV étanche avec des marqueurs Qualisys fixés. (Voir figure 4.9)

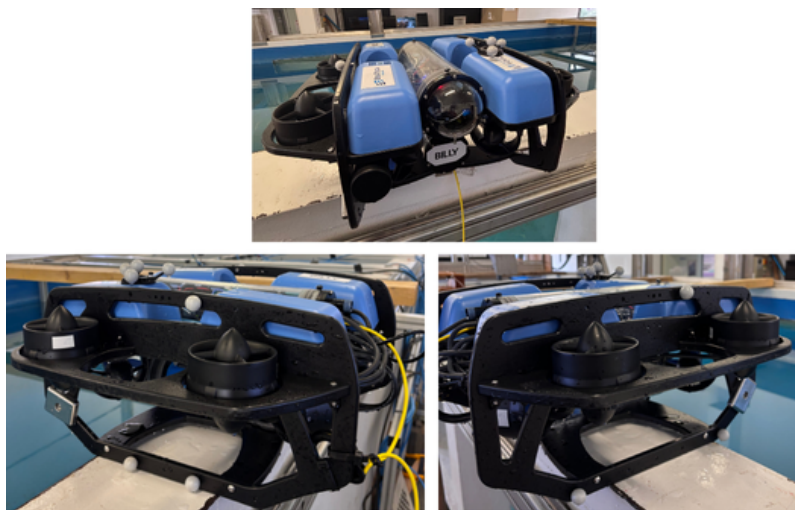


FIGURE 4.9 – ROV pour le bassin avec marqueurs Qualisys

- **Setup** : PC relié aux deux caméras. Support en palette pour les caméras avec de la corde. Le support est fixé avec des serre-joints au rebord du bassin. Grosse limite de distance à cause des câbles USB ce qui a impliqué de réduire la zone de travail. (Voir figure 4.10)

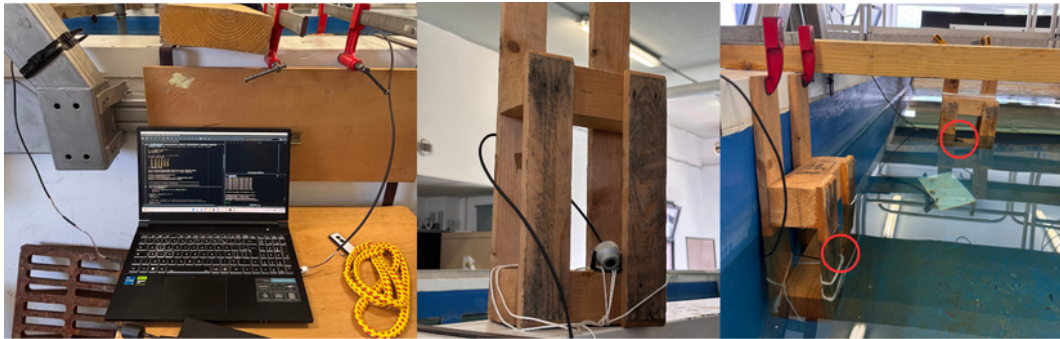


FIGURE 4.10 – Setup Bassin

- **Modification et maniement de la mire** : Nous avons percé les quatre coins de la mire pour pouvoir fixer un bout de corde à chaque coin et pour pouvoir la manipuler dans le bassin (Voir figure 4.11). De plus, sous l'eau, la résistance hydrodynamique rend la manipulation de notre mire de 85 cm très difficile. Bouger la mire trop vite créait des remous et du flou de mouvement sur l'image, empêchant OpenCV de détecter les AprilTags. Nous avons dû réaliser des mouvements lents pour valider le redressement du repère et l'étalonnage.

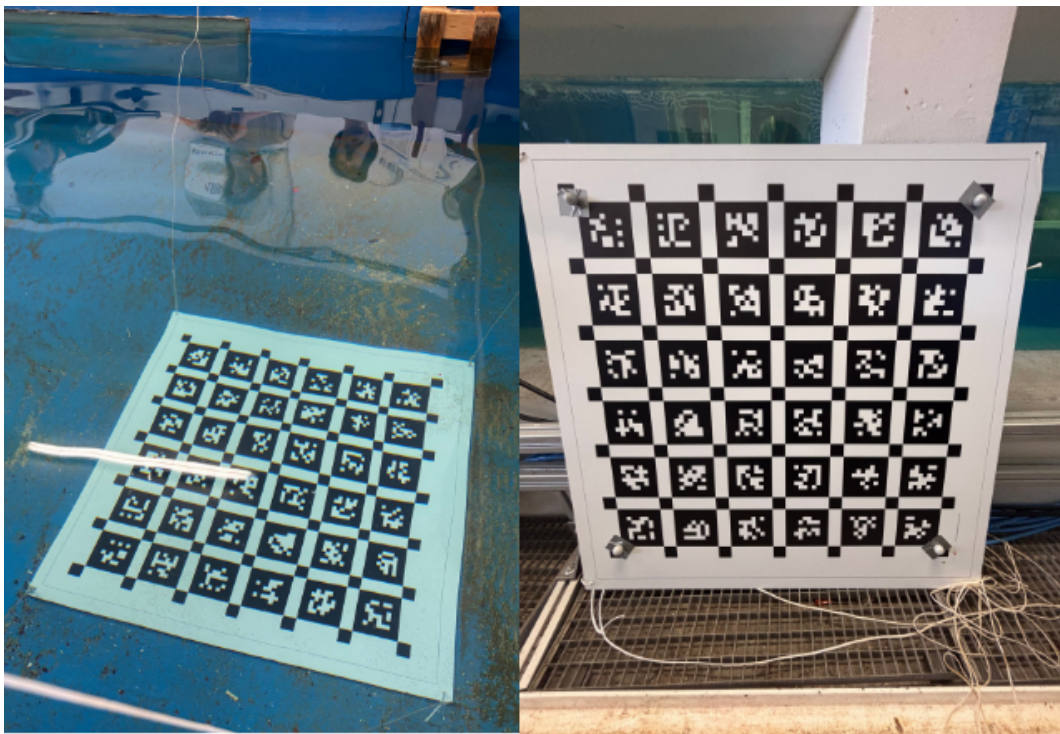


FIGURE 4.11 – Maniement de la Mire et Extrinsic Bassin

4.3.2 Analyse des Résultats - Test 1

Pour ce premier test en bassin, nous avons décidé de mesurer la précision de notre système en mesurant physiquement la distance entre les deux parois du bassin. Nous avons démarré l'acquisition lorsque le ROV était collé à une paroi, puis fait bouger le ROV jusqu'à la paroi en face. (Voir figure 4.12)

Comparaison par mesure

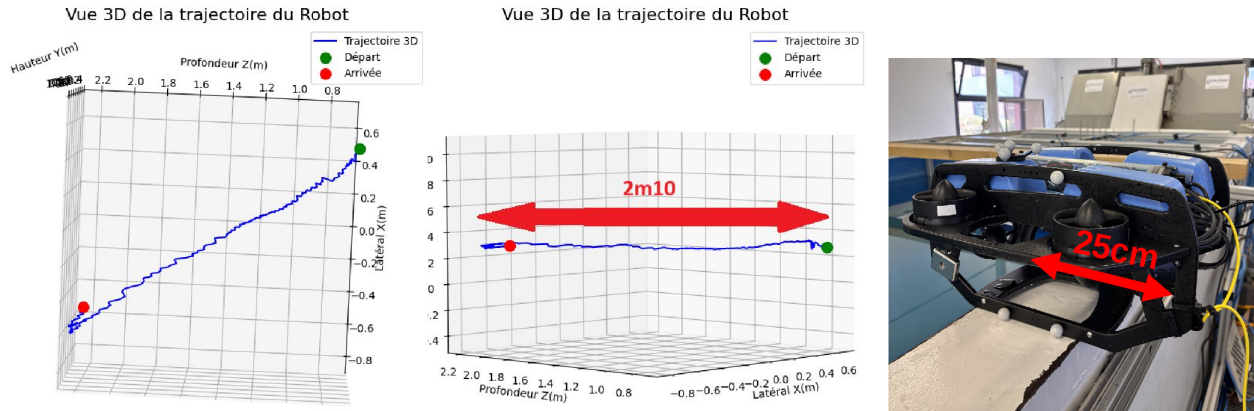


FIGURE 4.12 – Résultat Test 1

En sachant que la distance entre les deux parois intérieures du bassin mesure $2m60$ et que le barycentre du ROV est à $25cm$ du bout. La distance parcourue par le ROV calculée par notre système est de $2m10$, ce qui correspond bien à $2,60 - 2 * 0,25m$.

Cela permet de confirmer que nous avons les mêmes distances dans notre système que physiquement.

4.3.3 Analyse des Résultats - Test 2

Pour ce second test, nous avons comparé notre système au système Qualisys sur une manoeuvre simple : un virage dans une toute petite zone car nous étions limités par la zone de détection du Qualisys. (Voir figure 4.13)

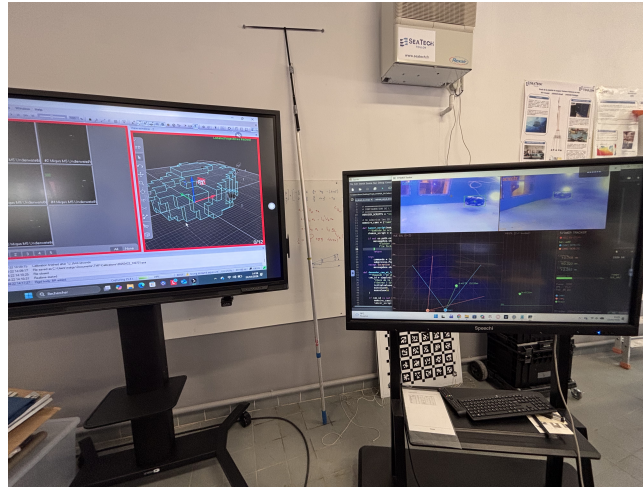


FIGURE 4.13 – Écran Qualisys - Écran IHM Tracking 3D

Comparaison avec le système Qualisys

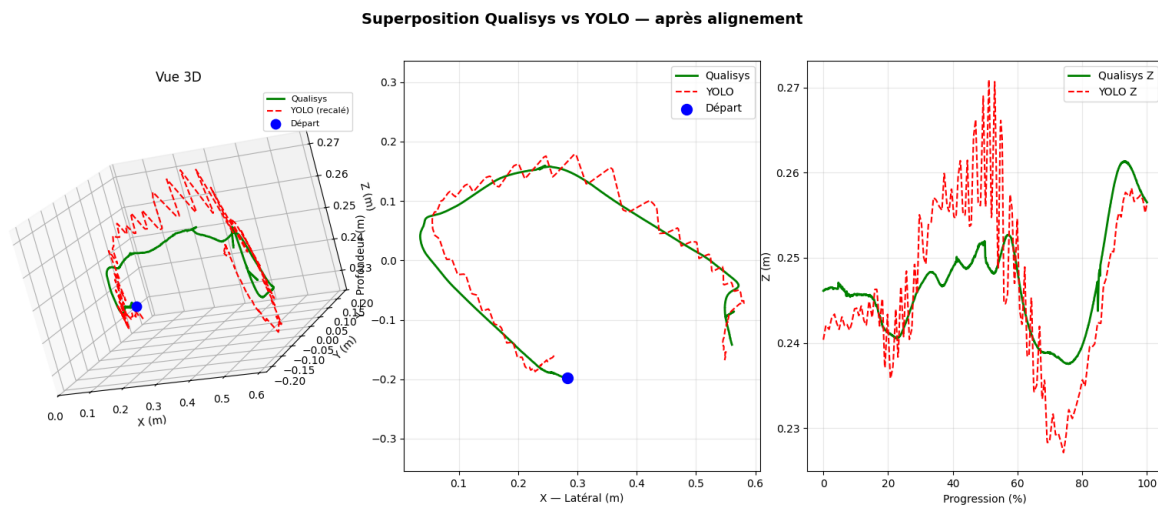


FIGURE 4.14 – Résultat Test 2

On remarque que notre système suit assez fidèlement la trajectoire du système Qualisys (Voir figure 4.14). En effet, malgré les petits sauts réguliers (bruit) de Yolo, l'échelle est suffisamment faible pour dire que notre système est précis et fiable.

Ces petites erreurs persistantes s'expliquent de la même manière que pour les tests dans l'air :

- Qualisys et notre système ne traque pas exactement le même point physique.
- La box de Yolo peut changer légèrement de taille ce qui modifie le calcul du barycentre.

On constate cependant que le passage de l'air au bassin n'a pas fortement dégradé notre système et sa précision.

4.3.4 Bilan

Résultats étalonnages :

Lors de l'étalonnage intrinsèque, nous avons eu un RMS de 1.31 pour la caméra 1 et 1.16 pour la deuxième, ce qui est très bon et confirme la validité de notre algorithme.

Pour l'étalonnage extrinsèque, notre système a trouvé 1m31 entre les deux caméras (Voir figure 4.15).

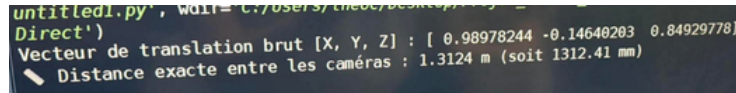


FIGURE 4.15 – Résultat étalonnage extrinsèque bassin

Physiquement, avec un mètre, nous avons trouvé le même résultat avec l'incertitude de la mesure physique. Cela permet de valider notre algorithme.

YOLO :

Lors des tests en bassin, notre détection 2D avec YOLOv8 a réussi à maintenir une boîte englobante stable autour du BlueROV. Cependant, à cause de la lumière violette des caméras Qualisys sous l'eau, nous avons dû rapidement ré-entraîner notre modèle YOLO.

Précision 3D

Concernant la précision 3D, nous retrouvons les mêmes limites que lors du test dans l'air : la trajectoire du robot est correctement reconstruite, mais elle conserve ce léger bruit (les sauts de pixels) dû à la variation de la taille de la *bounding box* YOLO d'une frame à l'autre. Le système remplit son rôle d'alternative légère et bas coût face au système Qualisys.

(L'analyse de la précision est disponible à la section suivante 4.4)

Limites de Qualisys

Dans le bassin, l'installation de Qualisys a été très complexe et longue. La zone de détection s'est révélée être très petite ce qui a fortement compliqué nos essais. De plus, comme le bassin n'était pas propre, les mouvements réalisés avec le ROV ont soulevé de nombreuses particules de sable et de poussière qui ont gêné la détection Qualisys car cela crée des faux points de tracking (Voir figure 4.16).

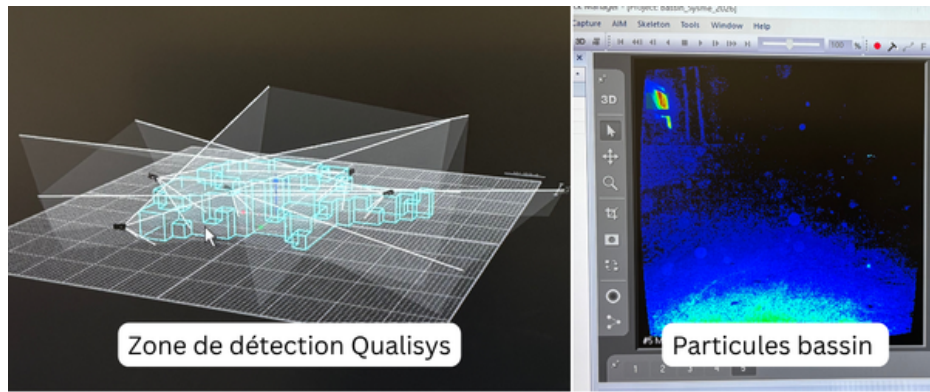


FIGURE 4.16 – Limites de Qualisys dans le Bassin

4.4 Analyse de la précision du système

La précision de notre système repose sur une chaîne d’acquisitions et de transformations mathématiques qui introduit des incertitudes. Nous avons identifié plusieurs sources d’erreurs :

- Les imprécisions lors de la calibration intrinsèque et extrinsèque, mesurables via l’erreur de reprojection RMS en pixels.
- L’incertitude lors du redressement du repère sur le sol du bassin.
- Le décalage temporel entre les flux de nos deux caméras USB.

Après analyse, l’erreur la plus importante vient des boîtes de YOLO. Pour comprendre ce problème, nous avons simulé l’observation d’un objet 3D par deux caméras.

(L’analyse de la précision GLOBALE du système avec tous ses détails est disponible en Annexe 6.8).

Finalement, le centre de la boîte 2D donnée par l’IA ne correspond jamais exactement au vrai centre 3D du robot. Ce décalage visuel varie selon l’angle de vue des caméras et fausse légèrement le calcul de triangulation. C’est une limite physique de notre méthode qui explique le décalage constant observé lors de nos tests.

(Pour une analyse correspondant à un cas pratique, veuillez aller à la dernière section du chapitre de l’analyse globale en annexe 6.8)

En bassin, la précision de notre modèle est donc toujours environ inférieur à $5cm$ voir dans les pires des cas $7cm$ ce qui est extrêmement correct.

5 Conclusion

L'objectif du projet de concevoir un système multicaméras standard de substitution à Qualisys est atteint. Le système détermine le positionnement 3D du BlueROV sans utiliser de marqueurs physiques avec une précision de l'ordre de la dizaine de centimètres. Les contraintes et les besoins, identifiés grâce à l'analyse fonctionnelle et organisationnelle, ont servi de cahier des charges. Au regard de ces exigences, le bilan du projet est positif. Nous avons pu tirer plusieurs conclusions lors de nos recherches :

- Les méthodes algorithmiques de vision classiques ne sont pas suffisantes pour de la détection sous-marine.
- La meilleure méthode de détection est celle par IA, via YOLOv8, qui est robuste et parfaitement adaptée au temps réel même si elle reste sensible aux variations d'éclairage.
- La calibration intrinsèque est essentielle, importante et peut-être source d'erreur si le RMS n'est pas suffisamment proche de 1.
- La précision de notre système de triangulation 3D est presque parfaite, avec une erreur semblable à celle de l'étalonnage extrinsèque.
- La précision globale dépend majoritairement de la détection YOLO du BlueROV, sa précision varie entre 4 et 10cm.
- Notre système multicaméras est moins précis que Qualisys dans l'air, avec une erreur dépendant de la précision de la détection.
- Notre système est légèrement moins précis que Qualisys dans l'eau, mais il est bien plus simple à mettre en place, plus rapide, moins coûteux en temps et en énergie et sa zone de détection est plus grande que celle du Qualisys, car les caméras de Qualisys détectent difficilement les marqueurs infrarouges dans le bassin.
- Les tests qualité, les mesures d'erreur, et de précisions sont cruciaux pour donner de la valeur à notre système.

Le produit final est structuré sous la forme d'un pipeline Python qui génère des fichiers de trajectoire directement exploitables par les chercheurs. Il est destiné à être repris, changé et amélioré. Les travaux pratiques et les codes sont documentés grâce aux différents manuels d'utilisation et ReadMe. L'ensemble des codes sont disponibles sur GitHub à l'adresse suivante :

<https://github.com/Projet-BlueROV-Seatech>

5.1 Respect du Cahier des Charges Fonctionnel

Le tableau de synthèse 5.1 confirme que les objectifs de notre système de positionnement multicaméras ont été atteints :

Réf.	Fonction (Critère attendu)	Résultat obtenu	Statut
FP1	Fournir la position 3D du BlueROV (<i>Erreur < 7 cm</i>)	Erreur d'environ 1,2 cm (à 1,5 m) et 4,7 cm (à 3 m). Pire des cas estimé à 7 cm.	Atteint
FC1	Architecture Python exploitable (<i>Codes et ReadME sur GitHub</i>)	5 dépôts GitHub publics fournis, commentés et documentés.	Atteint
FC2	Fonctionner avec des caméras standards (<i>Caméras USB grand public</i>)	Utilisation validée avec des caméras USB (DeepWater exploreHD).	Atteint
FC3	Résister aux perturbations visuelles (<i>IoU > 0.5</i>)	Détection très robuste avec un IoU proche de 1 via YOLOv8.	Atteint
FC4	Traiter le flux vidéo en temps réel (<i>> 15 fps sur CPU</i>)	Traitement fluide et continu du flux vidéo en direct via le pipeline.	Atteint
FC5	Ne pas modifier le robot (<i>Zéro marqueur ajouté</i>)	Suivi visuel sans aucun marqueur physique sur le ROV.	Atteint

TABLE 5.1 – Bilan du respect du Cahier des Charges Fonctionnel

5.2 Améliorations

Dans une optique d'amélioration continue et de perfection de notre rendu. Il est envisageable à l'avenir de :

- IHM pour YOLO : Il a fallu plusieurs fois ré-entraîner le modèle YOLO pour les tests dans l'air et dans le bassin avec Qualisys d'allumé (rayon bleu-violet). Pour améliorer et faciliter l'entraînement de l'IA, il faudrait créer une scène 3D du bassin et faire varier la position et l'orientation du BlueROV. Ce modèle serait parfait pour le bassin car l'environnement ne peut pas changer et il serait possible de changer la luminosité des images servant à l'entraînement selon les besoins.
- Changer la manière d'annoter sur YOLO : au lieu de faire un rectangle autour du robot, on pourrait utiliser d'autres types de boîte (cercle ou un triangle), ce qui permettrait d'avoir un barycentre plus précis et d'éviter d'encadrer l'environnement autour du ROV. Une autre méthode pourrait être d'entraîner YOLO uniquement sur le centre du robot, en mettant un point au centre afin qu'il s'entraîne à détecter le barycentre directement. On pourrait aussi ajouter des informations angulaires lors de l'apprentissage pour augmenter la précision de la détection et avoir la position 6D du ROV

sans la lourdeur de CosyPose.

- Changer de version de YOLO : des nouvelles versions de Yolo comme Yolo26 pourraient permettre d'améliorer fortement la précision du modèle et d'accélérer le processus d'entraînement.
- Rajouter un code pour optimiser la rotation et la translation entre notre caméra, la mire et le repère Qualisys en collant des marqueurs sur la mire. L'objectif est de pouvoir superposer au mieux les deux courbes obtenues par fichier TSV.

5.3 Ouverture

Il y a 2 projets qui sont dans la continuité de notre travail : l'intégration ROS et la création d'un environnement pour au moins 3 caméras.

Pour l'intégration du système sous ROS, il est possible d'utiliser le logiciel MAVLink qui permet de récupérer l'ensemble des canaux disponibles pour analyser l'architecture ROS. L'objectif est de faire du suivi de trajectoire et de pouvoir obtenir la position 5D (axe XYZ et les angles de roulis et de tangage). En s'abonnant à l'IMU (ou centrale inertielle) qui est composée d'un accéléromètre, d'un gyroscope et d'un magnétomètre, nous pourrions récupérer la mesure de la vitesse angulaire (taux de rotation) autour des axes. L'IMU risque de ne pas fonctionner car le robot est entouré de métaux dans le bassin, il faudrait alors le remplacer par une méthode de vision (marqueurs) ou acoustique (USBL, Ultra-Short Baseline qui calcule la position en mesurant à la distance (portée) et l'angle (gisement) par rapport à un transpondeur sous-marin).

Augmenter le nombre de caméras du système multicaméras pour affiner la position de la triangulation augmenterait considérablement la précision du système. Cependant, cela créerait un problème de synchronisation caméra dans les cas où le robot a un peu de vitesse. Il faudrait donc coupler le nouveau système multicaméras à un mini ordinateur de type Raspberry Pi pour pouvoir travailler avec de la stéréo vision.

Bibliographie

- [1] G. Bradski. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.
- [2] Z. Zhang. (2000). A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(11), 1330-1334.
- [3] S. Garrido-Jurado, R. Muñoz-Salinas, et al. (2014). Automatic generation and detection of highly reliable fiducial markers under occlusion. *Pattern Recognition*, 47(6), 2280-2292.
- [4] J. Redmon, S. Divvala, R. Girshick, et A. Farhadi. (2016). You only look once : Unified, real-time object detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779-788.
- [5] G. Jocher, A. Chaurasia, et J. Qiu. (2023). Ultralytics YOLOv8 (Version 8.0.0). [Logiciel]. Disponible sur : <https://github.com/ultralytics/ultralytics>
- [6] A. Kirillov, et al. (2023). Segment Anything. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 4015-4026.
- [7] Y. Labbé, et al. (2020). CosyPose : Consistent multi-view multi-object 6D pose estimation. *European Conference on Computer Vision (ECCV)*, 574-591.
- [8] M. Denninger, et al. (2019). BlenderProc. *arXiv preprint arXiv :1911.01911*.
- [9] S. Macenski, et al. (2022). Robot Operating System 2 : Design, architecture, and uses in the wild. *Science Robotics*, 7(66).
- [10] Qualisys AB. (2024). Qualisys Motion Capture Systems. Göteborg, Suède. <https://www.qualisys.com/>
- [11] A-W. Harouna Tiambou. (2025). *Segmentation d'objets et reconstruction 3D*. Mémoire, Université du Québec à Chicoutimi.
- [12] P. Virtanen, et al. (2020). SciPy 1.0 : fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261-272.
- [13] IJCV 47(1/2/3) :7-42, April-June 2002. Stereo Vision - Université de Zhejiang / Chapitre Erreur d'analyse (Page 15). <http://www.cad.zju.edu.cn/home/gfzhang/training/stereo/04stereo.pdf>

6 Annexe

Ces annexes servent de support aux lecteurs pour approfondir certaines notions abordées dans les chapitres précédents.

6.1 GitHub

Cliquez [ici](#) pour voir le projet hébergé sur GitHub.

6.2 Gestion de projet et Analyse Fonctionnelle

Retour au rapport 1.1.1

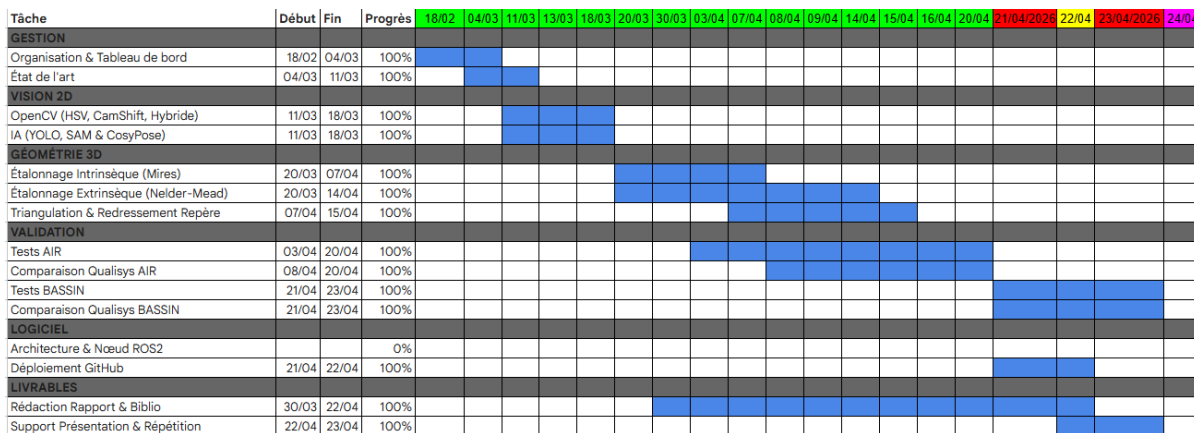


FIGURE 6.1 – Diagramme de Gantt illustrant la gestion des tâches dans le temps

Le diagramme a évolué avec le temps en fonction de l'avancé des progrès et de l'apparition de nouveau besoin. Nous sommes tout de même resté assez fidèle au fil conducteur que nous nous étions fixés en début de projet. Il y a cependant quelques incohérences entre le diagramme de Gantt et la réalité technique car nous avons plusieurs fois du modifier des anciens codes lié à la géométrie 3D et ré-entraîner YOLO.

MATRICE RACI				
Tâche	Théo	Timm	Julien	Romain
GESTION				
Organisation & Tableau de bord	R	R	R	R
État de l'art & Bibliographie	R	R	R	R
VISION 2D				
OpenCV (HSV, CamShift, Hybride)	A	R	R	A
IA (YOLO, SAM & CosyPose)	R	R	R	R
GÉOMÉTRIE 3D				
Étalonnage Intrinsèque (Mires)	R	A	R	A
Étalonnage Extrinsèque (Nelder-Mead)	R	R	A	R
Triangulation & Redressement Repère	R	R	A	R
VALIDATION				
Tests AIR & Comparaison Qualisys	R	R	R	R
Tests BASSIN & Comparaison Qualisys	R	R	R	R
LOGICIEL				
Architecture & Nœud ROS2	Annulé			
Déploiement GitHub	R	R	R	R
LIVRABLES				
Rédaction Rapport	R	R	R	R
Support Présentation	R	R	R	R

FIGURE 6.2 – Matrice RACI illustrant la répartition des tâches au sein de l'équipe

L'implémentation de notre système à l'architecture ROS2 a été annulée, en accord avec nos professeurs, à cause d'un manque de temps. La notation R correspond au statut de réalisateur et A à celui d'approbateur. Grâce à un système de roulement des posts, nous avons chacun pu travailler sur les différentes tâches et lorsqu'on ne pouvait pas, celles-ci nous étaient expliquées et l'on devait vérifier les travaux réalisés.

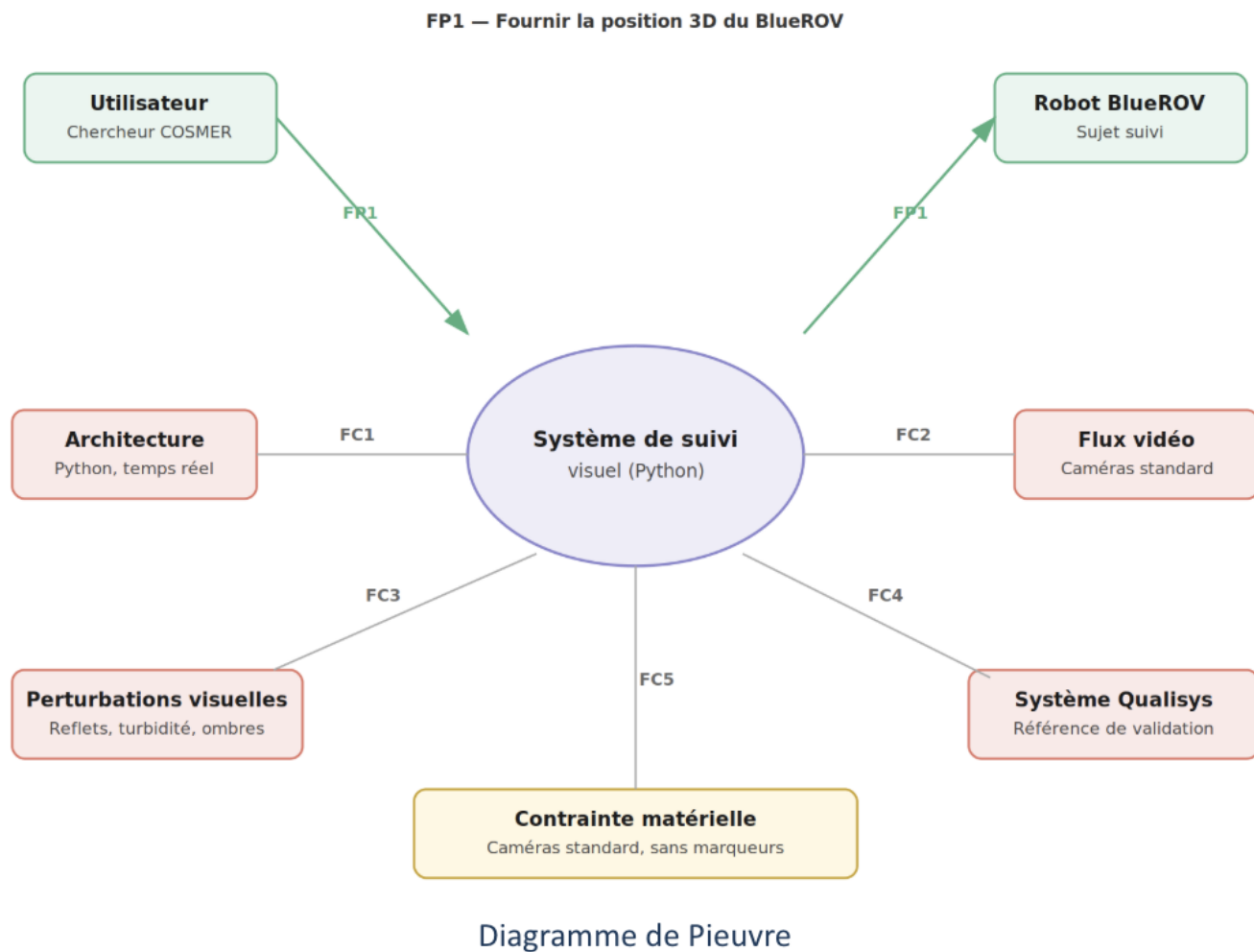


FIGURE 6.3 – Diagramme Pieuvre

Les éléments de ce tableau sont repris dans la suite du rapport. L'objectif de la figure est de pouvoir comprendre de manière rapide et succinct les enjeux du projet. Pour retourner au rapport cliquez ici 1.2.2.

6.3 Traitement d'images

6.3.1 Méthodes de Vision Classiques (OpenCV)

Cliquez [ici](#) pour voir les codes. (Retour rapport [2.1])

Notre objectif est de déterminer le centre et les bords du robot de manière algorithmique en utilisant les outils de la librairie OpenCV sur Python [1]. Le but de cette partie est de savoir si ces méthodes sont fiables et rapide.

6.3.2 Filtrage colorimétrique (HSV)

Notre première approche a consisté à isoler visuellement le ROV en se basant sur sa couleur (chassis sombre/noir). On a transformé les couleurs RGB en HSV (Teinte, Saturation, Valeur) pour mieux distinguer les variations de luminosité sous l'eau.

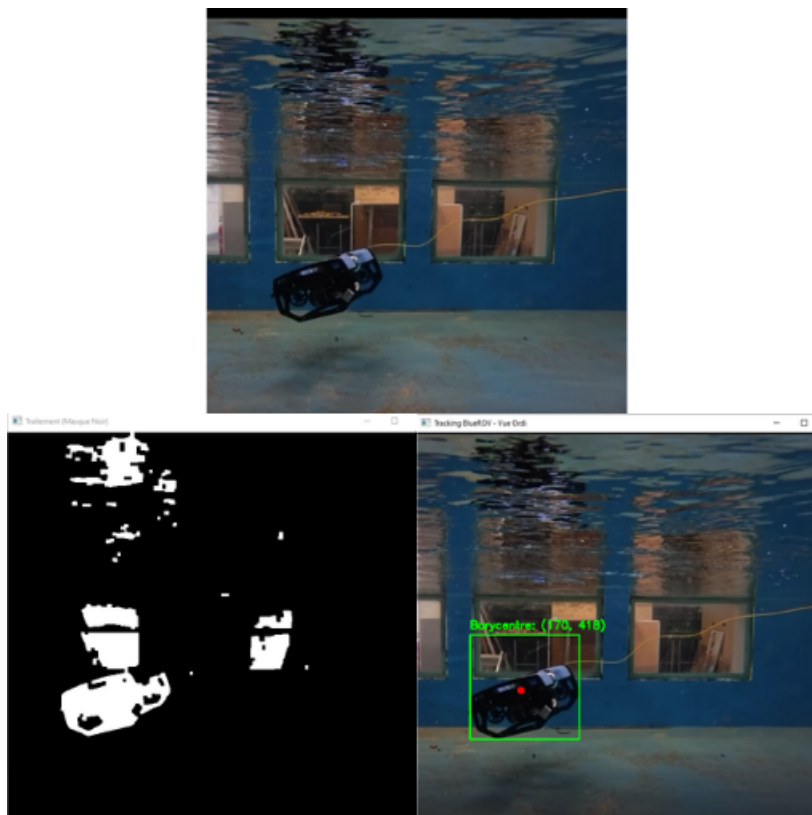


FIGURE 6.4 – HSV

On a ensuite appliqué un seuillage pour conserver uniquement les pixels sombres et on a éliminé les parasites visuels liés aux remous de l'eau avec des filtres morphologiques (érosion et dilatation). Une fois la masse sombre du robot encadrée par la bounding box (le plus grand contour), on trouve le barycentre (coordonnées (x, y) de son centre de gravité) en utilisant la fonction des moments spatiaux d'OpenCV (`cv2.moments`). (Le code est disponible [ici](#) [6.3.1])

Résultats et limites : Cette méthode a rapidement montré ses limites sous l'eau. Le filtre ne fait pas la différence entre le robot sombre et une ombre au fond de l'eau ou un reflet sombre à la surface, rendant le suivi trop instable.

6.3.3 Suivi par histogramme (CamShift)

On s'est donc tourné vers l'algorithme CamShift (*Continuously Adaptive Mean Shift*). Le programme demande à l'utilisateur de dessiner manuellement une boîte autour du robot sur la première image de la vidéo. Il calcule l'histogramme de cette zone précise.

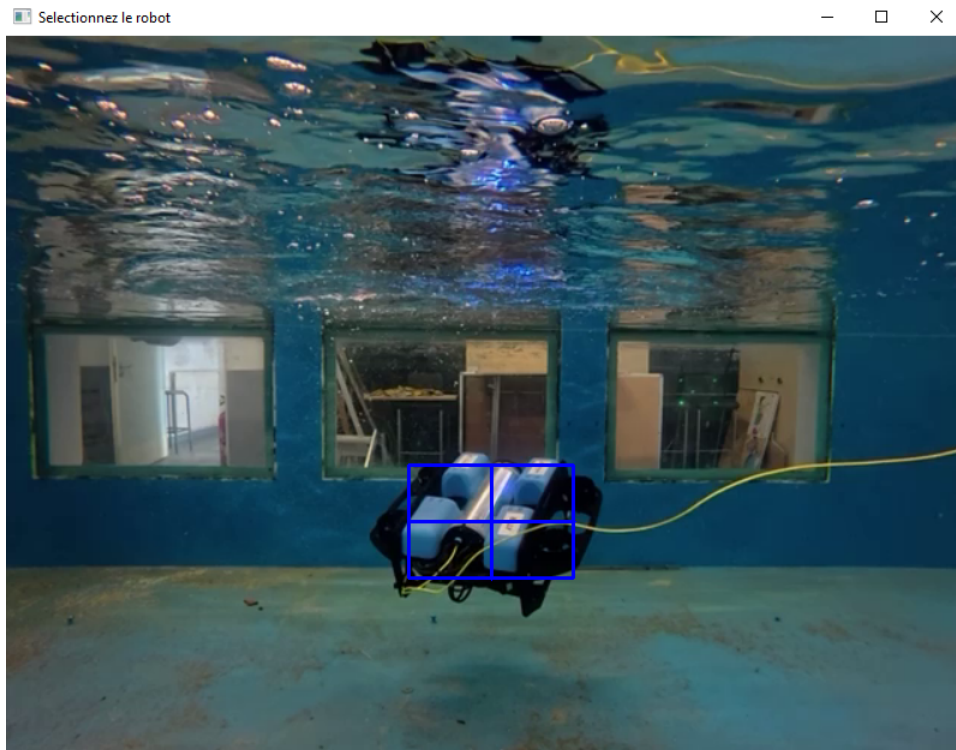


FIGURE 6.5 – Dessin manuel sur la première image de la vidéo

Sur les images suivantes, la fonction `cv2.CamShift` cherche où cette probabilité de couleur s'est déplacée, en adaptant dynamiquement la taille et l'inclinaison de la boîte de suivi en fonction de la distance et de la rotation du robot. (Le code est disponible ici 6.3.1)

Résultats et limites : Avec la profondeur (absorption du spectre lumineux), les couleurs varient trop par rapport à l'image initiale et l'algorithme perd sa cible très rapidement de manière irréversible. (Voir figure 2.1)

6.3.4 Détection par caractéristiques (ORB)

Nous nous sommes tournés vers l'algorithme ORB (*Oriented FAST and Rotated BRIEF*). L'idée est de fournir des images de référence du ROV (issue du modèle CAO) pour que l'algorithme extrait les points d'intérêts (angles, contrastes forts), puis il tente de retrouver ces points sur la vidéo. Si suffisamment de points géométriques correspondent (via le calcul d'une homographie), le robot est

encadré. (Le code se trouve ici 6.3.1)



FIGURE 6.6 – Images CAO de référence du ROV

Résultats et limites : L'algorithme ORB s'est montré totalement inefficace : les points d'intérêts du robot étaient noyés dans le bruit de l'image (reflets, remous), générant de fausses correspondances avec l'environnement autour du bassin.

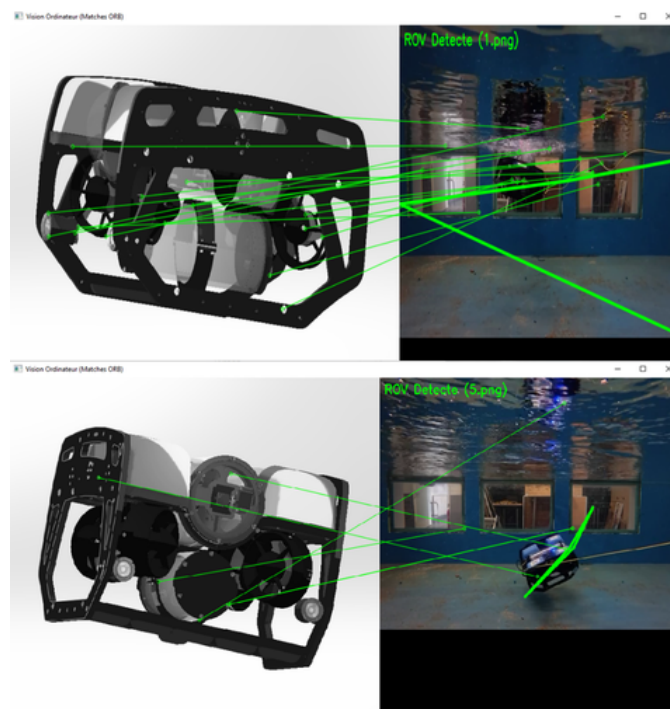


FIGURE 6.7 – Fausses correspondances de l'algorithme ORB

6.3.5 Stratégie hybride (HSV/ORB)

Pour tenter de résoudre les problèmes rencontrés avec l'algorithme ORB et le filtre HSV, nous avons développé une stratégie hybride. L'objectif était de créer un algorithme qui compense les faiblesses de l'autre. (Le code se trouve ici 6.3.1)

1. **Détection** : Utilisation d'un filtre HSV pour repérer une masse sombre importante (le châssis). Cela permet de définir une zone d'intérêt et d'éliminer le reste du bassin des calculs.
2. **Validation locale** : L'algorithme ORB est appliqué à l'intérieur de cette zone d'intérêt, il extrait les points d'intérêts et les compare avec nos images de référence (modèle CAO). Si le nombre de correspondances dépasse un seuil, la cible est validée comme le BlueROV.
3. **Suivi fluide** : Une fois la cible verrouillée, un algorithme de suivi d'objet intégré à OpenCV (Tracker MIL) prend le relais et suit le robot en temps réel de manière fluide, jusqu'à la prochaine validation de sécurité par ORB deux secondes plus tard.

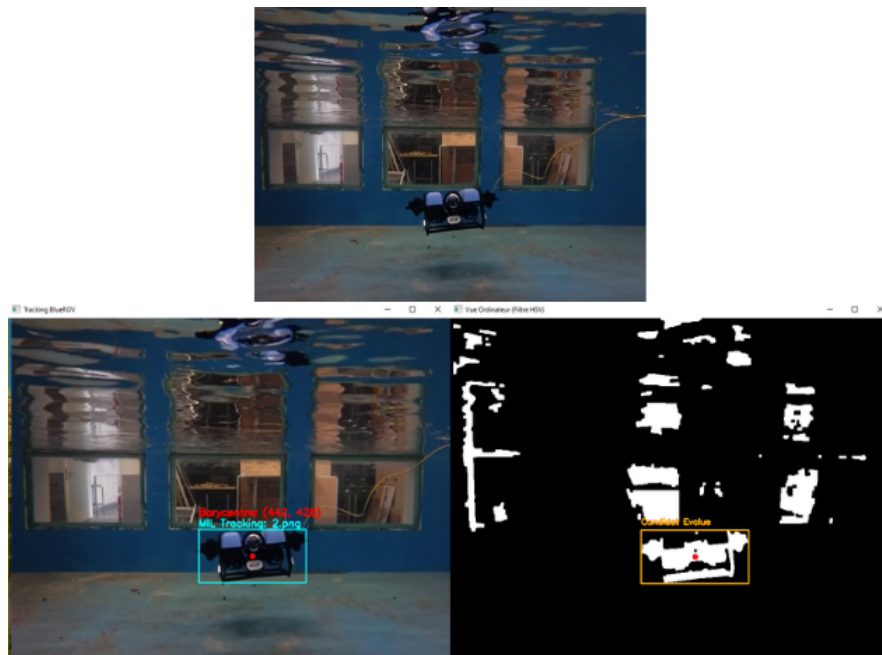


FIGURE 6.8 – HYBRIDE

Bilan général : pourquoi passer à l'IA ?

Malgré une méthode classique assez aboutie, l'environnement sous-marin a bloqué notre système hybride pour deux raisons :

- L'eau agit comme un filtre qui diminue fortement les contrastes et les textures.
- La coque du BlueROV étant lisse, l'algorithme ORB ne trouvait pas d'angles nets auxquels s'accrocher.

Sans ces points d'intérêts fiables, le système finissait toujours par perdre la trace du robot en glissant sur des reflets. Cet échec final de la vision classique a justifié pleinement notre passage à l'Intelligence Artificielle (YOLO). Contrairement aux anciennes méthodes, le Deep Learning est capable d'apprendre la forme globale du robot, même quand les détails et les contrastes manquent à l'image.

(Retour rapport [2.1])

6.4 Méthode par Intelligence Artificielle

(Retour rapport [2.2.1])

Notre objectif est de positionner dans l'image le centre du BlueROV en utilisant des méthodes de reconnaissance d'objets par IA. Le but de cette partie est de trouver une méthode fiable, dont la qualité peut être mesurée et qui fonctionne en temps réel.

6.4.1 YOLO v8

Tout d'abord nous avons décidé d'utiliser YOLOv8 [5], qui n'était pas entraînée à détecter le robot.

Entraînement

Pour utiliser l'IA, nous l'avons tout d'abord téléchargée sur un ordinateur afin de la faire fonctionner en local.

Pour l'entraîner, nous avons décomposé nos vidéos en extrayant une image (frame) toutes les 10 secondes. Nous avons ensuite annoté ces images à la main avec YOLO FiftyOne pour indiquer la position du robot (Voir figure 6.9). Afin de garantir la fiabilité de l'apprentissage, nous avons séparé nos données en deux catégories : un dossier d'entraînement (train) et un dossier de validation (val). Plus l'IA avait d'images pour s'entraîner, plus elle était performante pour la détection.

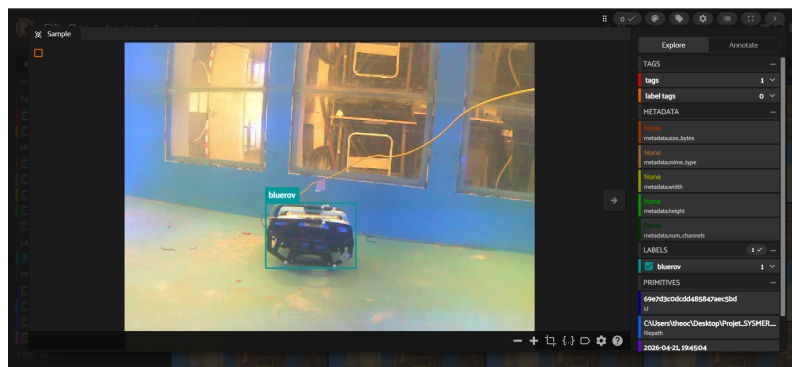


FIGURE 6.9 – Annotation d'une image avec YOLO FiftyOne

Cependant, durant ces entraînements, nous avons rencontré différents problèmes. Tout d'abord, à chaque fois que nous décidions d'ajouter des images, il était nécessaire de réentraîner l'IA afin qu'elle n'oublie pas les images précédentes, sachant qu'il fallait compter 10 minutes d'entraînement pour 1 000 images en utilisant le GPU de l'ordinateur.

De plus, nous avons remarqué que le modèle est très sensible à l'environnement : quand la luminosité change d'une vidéo à l'autre, l'IA peut ne pas reconnaître le robot. Il est alors nécessaire de réentraîner le modèle avec des images incluant ces nouvelles conditions lumineuses pour qu'il reste efficace.

Enfin, après ces entraînements, nous avons obtenu une IA assez performante pour détecter le BlueROV.

Détection

Nous avons ensuite utilisée notre modèle YOLO pour détecter le BlueROV sur les vidéos de nos essais. L'IA repère le robot en entourant celui-ci avec une boîte (bounding box). À partir de cette boîte, nous extrayons le point central en pixels, ce qui nous donne la position 2D du robot sur l'image. Cependant, durant cette phase de détection, nous avons rencontré différents problèmes.

Tout d'abord, l'environnement aquatique génère de nombreux reflets à la surface de l'eau. Dans certaines situations, l'IA confondait le vrai robot sous-marin avec son reflet à la surface, ce qui décalait la boîte englobante et faussait les coordonnées 2D extraites.

De plus, lorsque le robot n'est pas complètement visible sur l'image, le centre de la boîte ne correspond pas vraiment au centre du robot.

Enfin, malgré ces perturbations, nous avons réussi à obtenir un algorithme de détection 2D robuste et en temps réel, capable d'isoler de manière fiable les coordonnées en pixels du robot afin de les exploiter ultérieurement pour la modélisation mathématique.

Contrôle Qualité

Afin d'évaluer la qualité et la précision de nos détections, nous avons décidé d'utiliser la métrique d'Intersection sur Union (aussi appelée IoU). Cependant, une simple vérification visuelle à l'œil nu n'était pas suffisante, nous l'avons donc comparée à nos annotations.

Nous avons donc calculé l'IoU sur nos images de validation. Pour cela, la métrique mesure le chevauchement entre la boîte englobante prédite par l'algorithme YOLO et la boîte réelle que nous avons tracée à la main. Le calcul consiste à diviser l'aire de l'intersection des deux boîtes par l'aire de leur union. Plus les deux boîtes se superposent parfaitement, plus le score d'IoU s'approche de 1, ce qui traduit une détection très performante. (Voir figure 2.2)

(Retour rapport [2.2.1])

6.4.2 SAM

(Retour rapport [2.2.2])

Afin d'isoler le robot sur nos images de manière extrêmement précise, nous avons également envisagé d'utiliser l'IA SAM (*Segment Anything Model*) [6]. Contrairement à YOLO, qui se contente de tracer des boîtes englobantes (bounding boxes), SAM est un modèle de segmentation : il est conçu pour détourner n'importe quel objet au pixel près en générant un masque complet.

Nous avons donc étudié le fonctionnement de cette IA. Sa plus grande force réside dans sa capacité de généralisation : elle est capable de reconnaître et de détourner des objets complexes sans même avoir besoin d'être entraînée spécifiquement sur nos propres données. L'IA pouvait ainsi isoler les contours précis du BlueROV dès la première utilisation.

Cependant, durant nos recherches et nos tests, nous avons rencontré un problème technique bloquant. L'architecture mathématique de SAM est extrêmement lourde et demande une puissance de calcul considérable (Voir figure 6.10). Le temps de traitement nécessaire pour analyser une vidéo

s'est avéré beaucoup trop long, ce qui le rendait totalement incompatible avec notre besoin de suivre la position du robot en temps réel sur un flux vidéo.

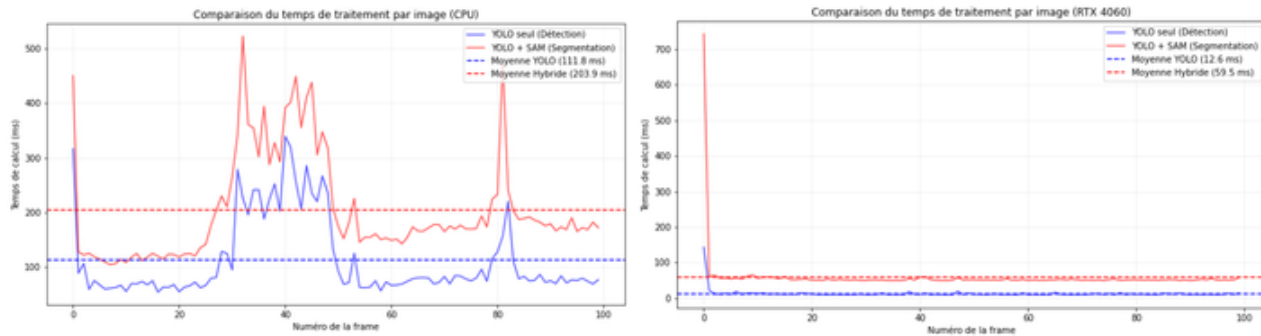


FIGURE 6.10 – Comparaison temps de calcul CPU vs GPU

(Retour rapport [2.2.2])

6.4.3 CosyPose

(Retour rapport [2.2.3])

Avant de nous concentrer sur YOLO, nous avons exploré une méthode d'Intelligence Artificielle beaucoup plus complexe : l'estimation de pose 6D avec des modèles comme CosyPose [7]. Sur le papier, cette méthode semblait être la solution à toutes les demandes du projet. Elle promettait de résoudre tous nos problèmes en nous donnant un maximum d'informations (la position en 6D : (X,Y,Z) et les 3 angles (yaw, pitch, roll) du robot).

Cependant, nous nous sommes très vite heurtés à la réalité de la mise en œuvre.

Le problème des données :

Pour entraîner un tel modèle à reconnaître le robot dans tous les sens et sous tous les angles, il faut générer un jeu de données (dataset) énorme. Notre premier blocage a été le modèle CAO du BlueROV, qui était beaucoup trop lourd à manipuler pour ce genre de calculs. Nous avons dû fouiller sur GitHub pour trouver un modèle 3D allégé, l'importer et le convertir dans *Blender*.

Ensuite, nous avons écrit un script sur *Google Colab* utilisant la bibliothèque *BlenderProc* [8] pour tenter de générer automatiquement des milliers d'images synthétiques du robot avec des éclairages et des angles aléatoires.

Comme le montre la figure 6.11, notre script parvenait à générer le modèle 3D en faisant varier les angles, mais dans un environnement vide. Si nous entraînions l'IA CosyPose sur ces données, elle serait incapable de reconnaître le vrai robot plongé dans le bassin d'essai à cause des perturbations visuelles (reflets, réfraction, fond bleu).



FIGURE 6.11 – Donnée générée via BlenderProc

(Retour rapport [2.2.3])

Un choix d'ingénierie :

Malgré nos efforts, nous avons réalisé que cette approche était un gouffre en termes de temps. Contrairement à YOLO, cette méthode exige une puissance de calcul énorme, elle est beaucoup moins évidente à mettre en place (problèmes de compatibilité des formats d'export, erreurs de scripts), et elle nous laissait très peu de contrôle sur le traitement de l'image lui-même.

Nous avons exposé nos difficultés à nos encadrants. Ils nous ont rassurés en nous confirmant que leur besoin principal pour le bassin d'essai était le suivi de la trajectoire en 3D (x, y, z) .

Nous avons donc fait le choix d'abandonner définitivement cette voie. S'entêter sur CosyPose s'avérerait trop complexe et peu rentable pour le temps dont nous disposions. Nous avons donc décidé de tout miser sur YOLO, une architecture plus facile à prendre en main, nécessitant moins de données, et dont les résultats étaient amplement suffisants pour valider notre projet.

(Retour rapport [2.2.1])

6.5 Modèles de positionnement 3D

6.5.1 Code étalonnage intrinsèque

(Retour rapport [3.1.1])

Notre script d'étalonnage se base sur la bibliothèque OpenCV et son module `cv2.ArUco`. On localise les coins la méthode `CORNER_REFINE_SUBPIX`. L'algorithme parcourt la vidéo et extrait automatiquement une image toutes les 45 *frames*. Chaque image nette doit contenir au moins 12 marqueurs pour être validée et pour que ses points soient enregistrés. Une fois un total de 20 poses distinctes accumulées (`NB_IMAGES_REQUISES = 20`), la fonction `cv2.calibrateCameraExtended` calcule la matrice intrinsèque de la caméra (`K.npy`), ses coefficients de distorsion (`D.npy`) et extrait les incertitudes (écarts-types).

(Retour rapport [3.1.1])

6.5.2 Étalonnage extrinsèque

(Retour au rapport 3.1.2)

L'étalonnage extrinsèque passe par plusieurs points clés, utilisant notamment les résultats de la calibration intrinsèque.

Etape 1 : rectification de la distorsion et étalonnage intrinsèque

Il y a une distorsion des images à cause de la courbure de la lentille. Les pixels sont mal placés et les lignes droites apparaissent courbées sur l'image. Pour corriger cela, on récupère les coefficients de la matrice de distorsion, obtenu lors de l'étalonnage intrinsèque, que l'on applique à l'image via une transformation non-linéaire.

Cette transformation se fait par le biais de la fonction `cv2.undistortPoints(pts, K, D, P=K)` d'Open CV avec `pts` les points déformés, `K` la matrice intrinsèque de la caméra, `D` la matrice de déformation de la lentille et `P=K` pour revenir aux paramètres d'origine de la caméra. On obtient alors une image redressé avec des pixels parfaitement placé.

Les 2 caméras n'ont pas les mêmes caractéristiques (résolution et focale) et les images ne font pas la même taille (car on est revenu aux paramètres initiaux `P=K`). Il faut pouvoir les normaliser en leur donnant les mêmes attributs. Pour cela, on passe sur un plan universel normalisé. On multiplie les coordonnées du pixel (ajoutées d'un 1 pour faire un vecteur 3D $[u, v, 1]$) par l'inverse de la matrice intrinsèque K^{-1} donné lors de l'étalonnage intrinsèque. La matrice K contient la focale (f_x, f_y) et le centre optique (c_x, c_y) :

$$x_{norm} = K^{-1} \cdot x_{pixel}$$

Dans notre code, les calculs sont exécutés par la fonction `normaliser_points(pts_px, K)`, on passe alors de pixels à des coordonnées normalisées sans dimension.

Etape 2 : Résolution analytique initiale (PnP)

L'objectif est de donner un point de départ à l'optimisateur de Nelder-Mead pour garantir sa convergence. Il existe 2 méthodes possibles, soit rentrer une approximation de la position de la caméra, soit utiliser un algorithme analytique `cv2.solvePnP` ce qui est plus optimal dans un système multi-caméras.

L'algorithme `cv2.solvePnP` met les deux caméras dans un repère monde commun. On définit d'abord la position absolue de la Caméra 1 (${}^wM_{c1}$), puis on calcule la position de la mire par rapport à cette caméra.

On connaît la géométrie de la mire, i.e. sa taille le nombre de lignes, de colonnes d'ArUcos et l'espacement entre les carrées. On détecte les identifiants propre à chaque ArUcos. On prend ensuite le centre des ArUcos projetées sur le plan image des caméras. (Voir figure 3.3)

Pour chaque point, on utilise la méthode itérative de `cv2.solvePnP` pour résoudre l'équation du modèle sténopé :

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} R & | & t \end{bmatrix} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix}$$

Avec :

- (X_w, Y_w, Z_w) : Les coordonnées 3D du centre des ArUcos.
- (u, v) : Les coordonnées en pixels sur l'image caméra.
- K : la matrice intrinsèque.
- s : Un facteur d'échelle pour ne pas perdre la notion de profondeur en projetant sur l'image).
- $[R|t]$: La matrice 3×4 que l'on veut déterminer, composée d'une rotation R et d'une translation t .

Le modèle procède de la manière suivante :

- Il fait une première estimation de R et t .
- Il projette mathématiquement les points 3D de la mire sur l'image en utilisant cette estimation.
- Il calcule l'écart (la distance en pixels) entre ces points projetés et les points réellement détectés sur l'image. C'est ce qu'on appelle l'erreur de reprojection.
- Il ajuste R et t pour minimiser cette erreur totale.

Les coordonnées 3D de la mire sont ensuite translatées (${}^{mire}X$) vers le repère monde pour obtenir de nouvelles coordonnées (wX). La fonction `cv2.solvePnP` est alors à nouveau appliquée sur les pixels 2D de la Caméra 2 en les liant à ses nouveaux points 3D du monde. Cette résolution nous donne une estimation mathématique fiable de la matrice homogène ${}^wM_{c2}$, représentant la pose absolue de la Caméra 2. La fonction `solvePnP_vers_matrice` converti les vecteurs de rotation et de translation générés par OpenCV en la matrice 4x4 :

$${}^wM_c = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}$$

Étape 4 : Optimisateur (Nelder-Mead)

La solution analytique du PnP est affinée par une optimisation non-linéaire (algorithme de Nelder-Mead avec `minimize` de SciPy [12]). L'objectif est de minimiser l'erreur quadratique de

reprojection : l'écart entre les coordonnées normalisées mesurées sur l'image et celles projetées par le modèle géométrique.

La fonction de coût (`cost_nelder`) s'écrit :

$$E = \sum_{i=1}^n \|x_{i,norm} - \pi({}^{c2}M_w \cdot {}^wX_i)\|^2$$

avec π : la fonction de projection sténopé normalisée.

L'initialisation avec les angles d'Euler et le vecteur de translation calculés par le PnP assure une convergence rapide vers le minimum global et évite le blocage dans des minima locaux.

Étape 5 : Extraction de la pose relative

Nous possédons les poses absolues optimisées des deux caméras dans le monde (${}^wM_{c1}$ et ${}^wM_{c2}$). L'objectif final de la calibration extrinsèque stéréo est d'isoler la matrice homogène relative ${}^{c2}M_{c1}$, c'est-à-dire la position et l'orientation exactes de la Caméra 2 en prenant la Caméra 1 comme origine absolue.

On calcule cette matrice de passage en multipliant la matrice inverse de la Caméra 2 (qui représente la pose du monde vu par la Caméra 2, notée ${}^{c2}M_w$) par la matrice absolue de la Caméra 1 :

$${}^{c2}M_{c1} = {}^{c2}M_w \cdot {}^wM_{c1}$$

À partir de cette matrice 4x4, on obtient :

- La rotation relative : La sous-matrice de rotation 3x3 (R) que l'on convertit en angles d'Euler extrinsèques (rx, ry, rz) via `Rot.from_matrix(R_rel_opt).as_euler(...)`.
- La translation relative : Le vecteur t nous donne les décalages spatiaux sur les 3 axes (X, Y, Z). En calculant sa norme euclidienne, on obtient la distance en mètres séparant les capteurs optiques des deux caméras.

(Retour au rapport 3.1.2)

6.5.3 Triangulation 3D

(Retour au rapport 3.2.1)

Pour suivre la position du robot en temps réel, nous avons écrit une fonction "triangulate_robots" qui prend en entrée les coordonnées des points sur les plans images des caméras post traitement et les matrices de passage de chaque caméra vers le repère P_1 et P_2 qui sont de la forme

$$P_1 = K[I|0]$$

et

$$P_2 = K.M$$

où M est la matrice de passage de la caméra 1 vers la caméra 2 et K est la matrice intrinsèque des caméras.

D'abord, la fonction transforme les coordonnées des points en vecteur colonne, puis elle utilise la fonction "triangulatePoints" de OpenCV. Les entrées de cette fonction sont :

- Les matrices de passage de chaque caméra vers le repère P_1 et P_2
- les points représentant le barycentre du BlueROV dans les plans image de chaque caméra pt_1 et pt_2 . Ces points sont de la forme

$$pt_1 = \begin{bmatrix} x_1 & y_1 \end{bmatrix}^\top$$

et

$$pt_2 = \begin{bmatrix} x_2 & y_2 \end{bmatrix}^\top$$

où x_1, y_1, x_2, y_2 sont les coordonnées des points dans les plans image de chaque caméra.

Grâce aux matrices P_1 et P_2 , il existe deux relations directes entre la position du barycentre sur les plans image de chaque caméra et sa position dans le repère 3D qui permettent de tracer les rayons partants du centre optique de chaque caméra et passant par le barycentre du BlueROV. (Voir figure 3.5)

Ces relations sont de la forme :

$$s_1 \cdot \begin{bmatrix} x_1 \\ y_1 \\ f = 1 \end{bmatrix} = P_1 \cdot \begin{bmatrix} X \\ Y \\ Z \\ f = 1 \end{bmatrix}$$

et

$$s_2 \cdot \begin{bmatrix} x_2 \\ y_2 \\ f = 1 \end{bmatrix} = P_2 \cdot \begin{bmatrix} X \\ Y \\ Z \\ f = 1 \end{bmatrix}$$

où s_1 et s_2 correspondent à la profondeur inconnue de la position de l'objet par rapport à l'une et l'autre des caméras, et f correspond à la focale des caméras, normalisée et donc égale à 1.

Après manipulation de ces équations, on obtient une équation matricielle de la forme

$$A \cdot X = 0$$

En résolvant cette équation, le script pourrait obtenir s_1 et s_2 et donc la position exacte du point de croisement entre les deux droites, qui correspond à la position du barycentre du BlueROV. Cependant, en raison du bruit numérique et des imperfections du modèle Yolo, les droites ne se croisent jamais parfaitement. C'est pourquoi, pour estimer la position du BlueROV, l'algorithme utilise la méthode des moindres carrés. Cette méthode consiste à trouver le milieu du plus court segment séparant un point de chaque droite et à utiliser le milieu de ce segment comme estimation finale de la position du BlueROV. Mathématiquement, cela revient à trouver $\min \|AX\|^2$.

La fonction renvoie alors un vecteur de dimension 4 de la forme $\begin{bmatrix} X & Y & Z & W \end{bmatrix}^\top$ où X, Y et Z sont les coordonnées homogènes du barycentre du BlueROV dans l'espace 3D et W est un facteur d'échelle.

Ici, les coordonnées sont exprimées en pixels, ce qui n'a pas de sens du point de vue de la réalité. C'est pourquoi notre script utilise la fonction "flatten" pour diviser chaque coordonnée par le facteur d'échelle W , et ainsi les exprimer dans la même unité que la matrice de passage d'une caméra à l'autre, ici en mètres.

Pour finir, la fonction "triangulate_robot" renvoie les coordonnées de la position du robot X , Y et Z exprimées en mètres, dans le repère 3D d'une des deux caméras.

(Retour au rapport 3.2.1)

6.5.4 Redressement repère

(Retour au rapport 3.2.2)

L'objectif du redressement est d'exprimer les coordonnées du barycentre du BlueROV dans un repère orthogonal au sol. Pour cela, nous avons écrit un script qui construit une matrice de redressement permettant de passer du repère caméra à ce nouveau repère à partir d'une image contenant une mire couchée sur le sol, prise par la caméra 1.

Après avoir détecté la mire au sol, le script corrige la distorsion grâce à la fonction "undistortPoints" puis utilise la méthode "solvePnP" pour créer un vecteur de rotation de Rodrigues "rvec_ole" et un vecteur de translation $tvec_sol$ qui définissent la position relative de la mire par rapport à la caméra 1. Ensuite, le script transforme le vecteur $rvec_sol$ en une matrice de rotation 3×3 R_sol grâce à la fonction "Rodrigues" de OpenCV puis il reconstruit chaque axe du repère orthogonal au sol X_world , Y_world et Z_world à partir de R_sol et $tvec_sol$.

— Axe Y (Verticale)

Pour construire l'axe Y, le script utilise la 3ème colonne du vecteur R_sol pour construire le vecteur n_up puis normalise ses composantes grâce à la fonction "linalg.norm" de la librairie numpy. Ensuite, le script effectue un test sur le produit scalaire $n_up.tvec_sol$. Si le signe de ce produit scalaire est positif, alors $Y_world = n_up$, sinon $Y_world = -n_up$.

— Axe X (Latéral)

Pour construire l'axe X, le script crée d'abord le vecteur $cam1_X$ qui correspond à l'axe X théorique de la caméra 1. Cet axe n'est en réalité jamais orienté parfaitement, c'est pourquoi le script le redresse en effectuant l'opération $(cam1_X.world_Y) * world_Y$ qui correspond à la projection de l'axe X théorique de la caméra 1 sur l'axe Y du repère orthogonal au sol. En déduisant ce résultat à $cam1_X$ le script obtient X_world qui est donc un axe parfaitement horizontal et orthogonal à Y_world et à l'axe Z de la caméra 1. Pour finir, le script normalise X_world .

— Axe Z (Profondeur)

Pour construire l'axe Z, il ne reste qu'à effectuer le produit vectoriel $X_world \wedge Y_world$ puis à normaliser le résultat pour obtenir Z_world .

Une fois tous les axes de ce nouveau repère construits. Ils sont empilés dans une matrice de redressement $R_redressement$.

Le nouveau repère créé a pour origine le centre optique de la caméra 1, or il est souhaitable que les hauteurs calculées lors du suivi correspondent à la hauteur réelle de l'objet. C'est pourquoi le

script effectue le produit matriciel $R_redressement \cdot tvec_sol$ pour obtenir le vecteur $sol_redresse$ dont la première composante est la hauteur de la caméra 1 $hauteur_cam1$. Cette distance sera utilisée dans la boucle de suivi pour calculer la hauteur réelle du barycentre du BlueROV.

Pour finir, le script recalcule la position de la caméra 2 dans le repère orthogonal au sol en effectuant le produit matriciel $R_redressement \cdot (-R_init^T \cdot T_init)$ ou R_init et T_init représentent la matrice de rotation et le vecteur de translation de la matrice de passage de la caméra 1 à la caméra 2 dans le repère caméra.

(Retour au rapport 3.2.2)

6.6 Manuel d'utilisation

(Retour au rapport 4.1)

Ce manuel présente les instructions pour utiliser le système de positionnement visuel multicaméras du robot BlueROV qui permet d'obtenir la position 3D (X, Y, Z) du robot en temps réel dans un bassin d'essai, en utilisant des caméras standards USB et YOLOv8, sans marqueur physique sur le robot.

6.6.1 Livrables — Dépôts GitHub

L'ensemble des livrables du projet est hébergé publiquement sur GitHub à l'adresse suivante : <https://github.com/Projet-BlueROV-Seatech>

Le projet est structuré en **cinq dépôts** distincts, chacun accompagné d'un ReadMe dédié :

Dépôt	Contenu
Methodes-Finales-YOLO	Scripts principaux du pipeline (Intrinsec, Extrinsec, Passage, Redressement, Tracking) en mode Direct (temps réel) et Post-Traitement (vidéo). C'est le dépôt principal à utiliser. Accompagné d'un ReadMe logiciel et d'un ReadMe d'installation physique.
IHM-Temps-Reel	Script principal qui lance automatiquement toute la chaîne (étalonnages + tracking + IHM) en une seule commande.
Modeles-YOLO	Modèles YOLO entraînés (best_air.pt , best_eau.pt) et scripts d'entraînement avec annotations FiftyOne. Permet de ré-entraîner le modèle sur de nouvelles conditions.
Methodes-Classiques-OpenCV	Codes des méthodes algorithmiques testées et écartées (HSV, CamShift, ORB, Hybride), correspondant à la section 6.3.1 du rapport.
Controle-qualite-stereovision	Scripts d'analyse de la précision globale du système (qualite_triangulation.py , precision.py), correspondant à la section 6.8 du rapport.

TABLE 6.1 – Dépôts GitHub du projet

Les ReadMe disponibles sur GitHub sont plus complets que ce manuel : ils couvrent l'installation de Python et de l'IDE (Anaconda/Spyder), la configuration des environnements conda, la structure détaillée des dossiers et les résolutions des problèmes fréquents.

6.6.2 Prérequis et Matériel

Matériel nécessaire

- **Caméras** : 2 caméras USB (ex : DeepWater exploreHD).
- **Mire d'étalonnage** : Une mire avec des AprilTag 36h11 (grille de 6×6 , côté du carré $L = 8,8$ cm, espacement $S = 2,8$ cm).
- **PC** : Avec 2 ports USB et les librairies installées.
- **Robot** : Le BlueROV.
- **Support** : Pour pouvoir fixer les deux caméras.
- **Cordes** : Pour bouger la mire dans le bassin lors de l'étalonnage.

Logiciels et Librairies (Python 3)

Assurez-vous d'avoir installé les librairies suivantes via votre terminal :

```
1 pip install opencv-python opencv-contrib-python numpy scipy matplotlib pandas ultralytics
```

Fichiers requis

- Tous les scripts Python (`Intrinsec.py`, `Extrinsec.py`, `Passage.py`, `Redressement.py`, `Tracking.py`).
- **Le modèle IA** : Le fichier de poids YOLO (`best.pt`) issu de l'entraînement sur le BlueROV.

AVANT DE COMMENCER : Modifiez les chemins absolus (`DOSSIER_CALIB`, `CHEMIN_VIDEO` et chemin vers `best.pt`) pour qu'ils correspondent à l'emplacement de vos fichiers sur votre PC.

6.6.3 Pipeline d'utilisation (Étape par étape)

Étape 0 : Installation physique des caméras

1. Fixez les deux caméras approximativement à la même hauteur et orientées légèrement vers le bas en direction de la même zone de travail (orthogonale).
2. L'angle entre les axes optiques des deux caméras **ne doit pas dépasser 90°**. Si l'angle est trop grand, les calculs mathématiques pour l'étalonnage extrinsèque échoueront.
3. Branchez les caméras à l'ordinateur. Trouver leurs identifiants USB avec le code (`Test_index`) pour modifier le code principal.

Étape 1 : Étalonnage Intrinsèque (`Intrinsec.py`)

Ce code corrige la courbure de la lentille (effet fisheye) pour chaque caméra de manière indépendante. **À faire une fois pour la Caméra 1, une fois pour la Caméra 2**). N'oubliez pas de modifier les variables `ID_CAM`, `FICHIER_K_OUT` et `FICHIER_D_OUT` dans le script entre les deux passages.

1. Lancez `Intrinsec.py`.
2. Déplacez la mire devant la caméra uniquement entre les photos que le script capture automatiquement. Si la mire bouge pendant la capture, le flou de mouvement fera baisser la précision.

3. Inclinez la mire et placez-la dans **tous les coins** de l'image.
4. Le script s'arrête tout seul après 20 captures valides.
5. **Contrôle Qualité** : Dans le terminal, si l'erreur RMS est inférieure à 1.5 pixels, vous pouvez utiliser les matrices (`K1.npy`, `D1.npy`...). Mais, si le RMS est trop élevé, filmez à nouveau en variant davantage les angles et les distances.

Étape 2 : Étalonnage Extrinsèque (`Extrinsec.py`)

Ce code calcule la distance exacte et l'angle entre vos deux caméras.

1. Lancez `Extrinsec.py`. Les deux flux vidéo s'affichent.
2. Placez la mire dans la zone d'intersection pour qu'elle soit **vue par les deux caméras en même temps** (Vérifiez que l'angle entre les deux caméras est inférieur à 90°).
3. Assurez-vous d'avoir au moins 20 marqueurs AprilTags détectés en commun.
4. **Immobilité** : Maintenez la mire parfaitement immobile. Appuyez sur la touche ENTRÉE de votre clavier.
5. Le code va analyser 10 frames consécutives pour s'assurer de la stabilité.
6. Une fenêtre 3D va s'ouvrir pour vous montrer la position estimée des caméras.
7. Le terminal vous demande si le résultat est satisfaisant. Si la distance affichée correspond à la distance physique mesurée entre les deux caméras avec une très faible erreur, tapez 0 (Oui). Le script sauvegarde `R_c2_c1.npy` et `t_c2_c1.npy`. Si l'erreur est supérieure à 4cm, recommencez. Depuis `Lanceur_IHM.py`, il est possible que l'affichage dépendant des scripts ne s'affiche pas selon les IDE python.

Étape 3 : Matrices de Projection (`Passage.py`)

Cette étape purement algébrique prépare les matrices finales pour la triangulation.

1. Lancez simplement le script `Passage.py`.
2. Le code va lire les fichiers générés aux étapes 1 et 2.
3. Il génère automatiquement `P1.npy` et `P2.npy` dans votre dossier.

Étape 4 : Redressement du repère (`Redressement.py`)

Pour que la hauteur Z calculée corresponde à la vraie profondeur du bassin, le système doit savoir où est le sol.

1. Posez la mire d'étalonnage **bien à plat au fond du bassin** devant la caméra de votre choix.
2. Lancez `Redressement.py`.
3. Lorsque le terminal affiche la détection, appuyez sur Entrée.
4. Vérifiez l'erreur de reprojection affichée (doit être inférieure à 0,5 pixel).
5. Le script de redressement détectera cette image au démarrage, calculera l'horizontale et la verticale parfaites, et créera la matrice `R_redressement.npy` tout seul qui sera ensuite utilisée dans le tracking.

Étape 5 : Suivi 3D en Temps Réel (Tracking.py)

Votre système est étalonné. Vous pouvez maintenant suivre le robot.

1. Plongez le BlueROV dans l'eau.
2. Lancez `Tracking.py`.
3. L'Interface Homme-Machine (IHM) s'ouvre (Voir figure 6.12) . Vous y verrez :
 - Les flux vidéo avec les boîtes de détection YOLO.
 - Une carte vue de dessus (Plan X-Z).
 - Une carte vue de profil (Hauteur Y).
 - Un tableau de bord avec les coordonnées (X, Y, Z) en mètres.
4. Appuyez sur **ESPACE** pour mettre en pause l'enregistrement de la position, ou sur **Q** pour quitter.



FIGURE 6.12 – IHM Suivi 3D

6.7 Post traitement : Comparaison Qualisys

Cette section est réservée à l'analyse des performances mathématiques de notre système face à la vérité terrain (système Qualisys de capture infrarouge).

Comparaison graphique (TSV_Rota.py)

Ce script compare mathématiquement le fichier TSV obtenue avec le fichier TSV exporté depuis l'ordinateur Qualisys.

- **Ce qu'il fait** : Puisque les deux systèmes ne démarrent pas exactement à la même milliseconde, le script utilise un algorithme d'optimisation (*Nelder-Mead*) sur le début de la trajectoire pour aligner parfaitement les courbes dans l'espace et dans le temps.
- **Sortie** : Il génère une fenêtre avec 3 graphiques (Vue 3D, Vue de dessus, Évolution de la hauteur) et calcule les erreurs **RMSE (Root Mean Square Error)** en mètres sur chaque axe. Cette erreur est pour chacun des points. Autrement dit si Qualisys est démarré plus tôt et enregistre une courbe en plus, alors il aura plus de points que notre modèle et l'erreur sera injustement très élevé.

Afin de comparer les deux courbes, il faut faire varier la matrice de rotation `R_yolo_to_qualisys` du modèle. Celle-ci n'était pas la même lors des tests en air et en bassin car le repère associé à Qualisys différait.

(Retour au rapport 4.1)

6.8 Analyse de la précision du système

(Retour au rapport 4.4)

Il est important d'évaluer l'erreur introduite à chaque étape préliminaire. L'objectif de cette section est d'évaluer la précision globale du système en analysant les différentes sources d'incertitudes.

Étalonnage intrinsèque

La validité de l'étalonnage est jugée selon la précision des indicateurs affichés :

- L'erreur RMS (Root Mean Square) : mesure l'erreur de reprojection. C'est la distance en pixels entre la position exacte du coin détecté sur l'image et la position où l'algorithme pense que ce coin devrait être en utilisant les paramètres qu'il vient de calculer.
- L'erreur par image : correspond au RMS de chaque image et permet d'écarter celles qui viendraient fausser le calcul lors d'un post traitement éventuel.
- Les fichiers d'incertitudes : correspondent aux écarts-types des paramètres de la matrice intrinsèque et de distorsion, i.e. c'est les marges de doute sur les données de distance focale, de centre optique et de coefficients de distorsion (exemple : si K a une erreur de 8 pixels alors sur une focale situé à 800 pixels, on est sûr avec nos mesures que la focale est environ à 8 pixels de la valeur 800). Avec peu d'images différentes de la mire, l'algorithme va deviner certaines valeurs et l'incertitude sera élevée.

Lors de l'étalonnage se baser sur la RMS est largement suffisant. Les incertitudes servent en post traitement pour connaître de manière très précise les limites du système.

Étalonnage extrinsèque

La précision dépend de l'erreur de reprojection, exprimée en pixels.

- $err1$: est l'erreur géométrique entre la mire et la caméra 1. Il s'agit de l'écart moyen en pixels entre les points détectés sur la photographie de la caméra 1 et la projection mathématique de la mire optimisée.
- $err2$: l'erreur entre la mire et la caméra 2, calculée de manière identique pour la seconde caméra.

L'erreur mesurée est d'environ $0.2px$, au-dessus de 0.5 on estime que ce n'est plus correct. Aucune erreur n'est mesurée directement entre la caméra 1 et la caméra 2 en pixels, car les caméras ne se voient pas mutuellement, elle est uniquement déduite de l'observation de la mire à la même date.

Reconstruction algébrique

Les matrices de passage n'introduisent pas d'erreur supplémentaire car c'est uniquement de l'algèbre.

Redressement du repère

La variable err_sol correspond à l'erreur de reprojection moyenne (en pixels) calculée lors de l'estimation de la pose avec la mire posée au sol. Le calcul est semblable à celui de l'erreur lors de l'étalonnage extrinsèque. Cette valeur mesure donc l'écart moyen entre :

- Les points 2D observés qui sont les centres des ArUcos détectés après correction de la distorsion.
- Les points 2D projetés : la position mathématique où ces points devraient s’afficher sur l’image si l’on se fie à la pose calculée par l’algorithme.

Cette erreur est du même ordre que celles de l’étalonnage extrinsèque. Plus cette valeur est faible (en dessous de 0.5 pixel), plus l’orientation calculée du plan horizontal (et donc vertical) dans le monde 3D est précise.

Détection YOLO et triangulation

Lors du suivi du BlueROV, plusieurs actions créent des erreurs, leur mesure s’effectue en quatre étapes :

- La détection YOLO : le modèle détecte le robot et extrait le centre géométrique des boîtes englobantes sur la caméra 1 (p_1) et la caméra 2 (p_2).
- La triangulation 3D : à partir des deux points 2D et des matrices de projection, l’algorithme calcule la position 3D la plus probable.
- La reprojection : l’algorithme effectue le chemin inverse en projetant ce point 3D sur les plans images des caméras 1 et 2 pour obtenir ses coordonnées théoriques ($p2d_1$ et $p2d_2$).
- Le calcul de l’erreur : l’erreur finale est la distance euclidienne en pixels entre le centre de la boîte YOLO observée et le nouveau point projeté.

L’erreur obtenue par triangulation (semblable à celle de l’étalonnage extrinsèque) est comprise dans cette valeur mais est largement inférieur aux sauts de la boîte de YOLO. Il faut donc se concentrer sur la détection.

Cette erreur de reprojection YOLO est plus élevée que celle obtenue avec les cibles servant à l’étalonnage car contrairement à un marqueur ArUco plat et parfaitement défini, le robot est un objet en volume.

YOLO détecte une face du robot, en raison des différents points de vue des caméras, les rayons de projection ne se croisent pas parfaitement. L’algorithme de triangulation approxime un point 3D par minimisation. En reprojétant ce point intermédiaire, on obtient un décalage en pixels par rapport aux centres YOLO bruts de départ.

Ainsi, une erreur faible signifie que YOLO vise exactement la même coordonnée physique du robot sur les deux caméras. Dans le cas contraire, on a des sauts de pixels ce qui crée un décalage observé sur la trajectoire 3D finale.

Plus les caméras voient le ROV avec une perspective non symétrique par rapport à un axe, plus il y a d’éléments différents en profondeur de l’objet encadrés et plus le barycentre est décalé. En revanche, si les caméras voient uniquement une face de l’objet ou bien l’objet avec une profondeur égale, l’estimation du barycentre devient précise.

Contrôle qualité de la mesure d'erreur de détection et de triangulation

Le calcul de l'erreur de la détection YOLO et de la triangulation n'est pas vraiment précise car elle ne prend pas en compte le véritable barycentre du ROV.

Afin de pouvoir mesurer l'impact d'un décalage en pixel lors de la projection, nous simulons un environnement avec un pavé droit et 2 caméras. Nous allons mesurer la variation angulaire de l'erreur en pixels des points projetés sur le plan image du barycentre du ROV et de la détection YOLO.

Le code python `qualite_triangulation.py` dont la structure a été générée par IA, puis adaptée, sert à évaluer cette erreur. Il est disponible à la page suivante : <https://github.com/Projet-BlueROV-Seatech/Controle-qualite-stereovision>

Nous encadrons le pavé droit observé sur le plan image des caméras afin de simuler YOLO. On récupère ensuite le centre des boxes qui servent pour la triangulation 3D. On affiche le point obtenu via la triangulation dans le monde 3D.

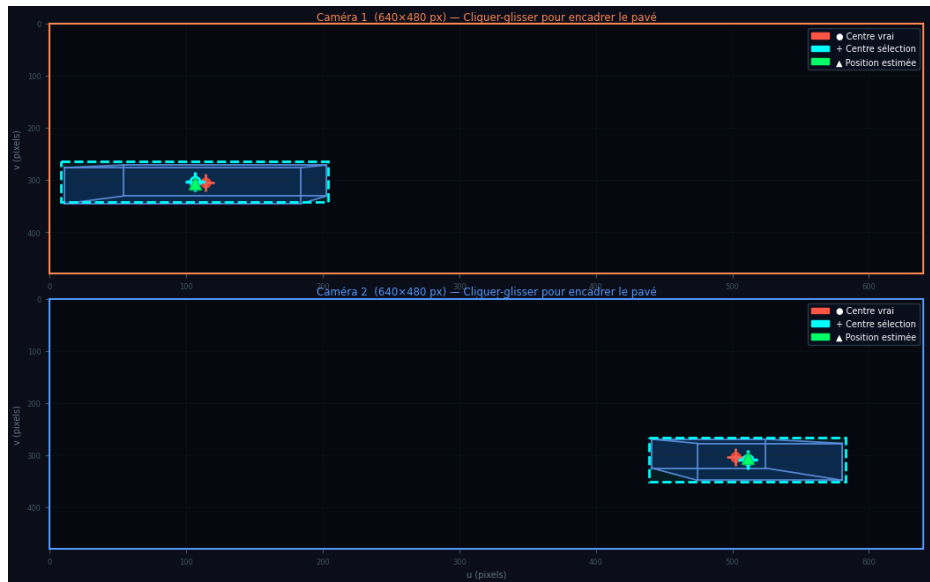


FIGURE 6.13 – Simulation manuelle de YOLO

Sur l'image 6.13, le point rouge représente le barycentre du ROV et le point vert est le centre supposé du ROV. On obtient ensuite la position du point vert dans le monde 3D.

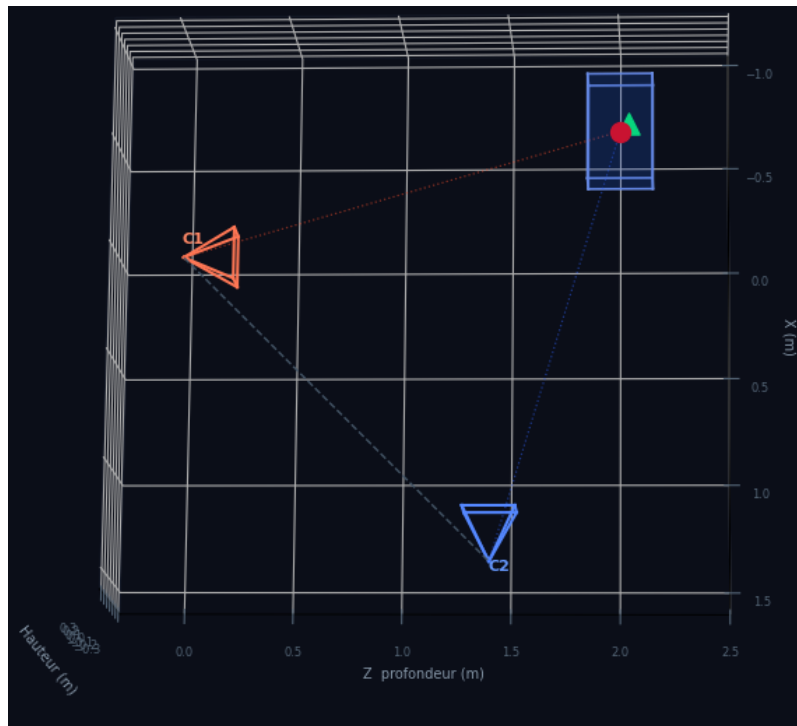


FIGURE 6.14 – Triangulation par stéréovision

Sur l'image 6.14, la caméra orange correspond à la cam1 et la caméra bleu à la cam2. Cette image sert à visualiser l'erreur de distance entre les deux points.

Nous mesurons la distance entre les pixels rouge et vert sur les plans images des caméras et l'angle des deux droites entre la focale et les points 2D.

```

Position vraie      : X=-0.6440 Y=+0.2060 Z=2.0000 (m)
Position estimée   : X=-0.6809 Y=+0.2136 Z=2.0399 (m)
Erreur 3D euclidienne : 54.823 mm
Cam1 – erreur angulaire : 0.61008° | distance : 7.61 px
Cam2 – erreur angulaire : 0.74176° | distance : 9.06 px
  
```

FIGURE 6.15 – Valeur des erreurs

Le code rend l'image 6.15, l'erreur est d'environ 5 – 6cm. Les distances en pixels séparant les deux points sur les plans images sont faibles. Avec une erreur angulaire toujours inférieur à 1° , on s'attend à ce qu'en 3D, l'erreur de projection du barycentre estimée ne dépasse pas les 7cm à 3m. La précision de notre système est donc de 1 degrés d'angle. Cette précision est plus affectée par la variation de perspective de l'objet que par la distance. Avec un modèle YOLO différent de détection, il serait possible d'avoir une estimation du centre du ROV presque parfaite.

Code d'analyse d'erreur

De cette section sur l'analyse de l'erreur globale résulte le code `precision.py` qui permet de récupérer chacun des fichiers d'**erreur** enregistré lors de la chaîne d'acquisition et de mesurer l'incertitude ou erreur globale en pixel (noté `erreur_globale_px`). Nous supposons que les différentes erreurs sont parfaitement indépendantes et qu'elles ont le même impactes sur l'image de manière isométrique (en réalité les propagations géométriques d'erreurs ne se font pas de la même manière). La formule est :

$$\sqrt{\left(\frac{K1 + K2}{2}\right)^2 + \left(\frac{\text{extrin}_{\text{cam1}} + \text{extrin}_{\text{cam2}}}{2}\right)^2 + \text{redressement}^2 + \text{triangu_et_yolo}^2}$$

Attention il y a plusieurs valeurs à corriger avant de lancer le script :

- la distance extrinsèque obtenue lors d'un contrôle qualité via Qualisys, noté `valeur_qualisys` en *mm*. S'il n'y a pas de contrôle qualité disponible, il faut donner à cette valeur la même valeur que celle obtenue par l'étalonnage extrinsèque `dist` pour ne pas fausser les calculs d'incertitudes qui suivent.
- les distances de détection, noté `Z_distance_m` afin de connaître la borne min et max de l'incertitude du système.

La valeur d'incertitude en millimètre est obtenue via l'erreur globale en pixel, via la formule :

$$\Delta Z = \frac{Z^2}{f \cdot B} \cdot \Delta d$$

Avec Z : la distance de l'objet en mm, f : la distance focale de la caméra en pixels, B : la distance entre les deux caméras en mm et d : la disparité (décalage entre les deux images en pixels). (Voir [13])

Nous allons maintenant nous intéresser au deuxième test réalisé en bassin.

```

🔦 BILAN DE CORRELATION ET PRECISION GLOBALE DU SYSTEME
=====
[1] INTRINSÈQUE : Incertitude de projection CAM1 : 3.1261 px
[1] INTRINSÈQUE : Incertitude de projection CAM2 : 2.4883 px
[1] INTRINSÈQUE : Distorsion CAM1 : 0.0436 px
[1] INTRINSÈQUE : Distorsion CAM2 : 0.0365 px
[2] EXTRINSÈQUE : Erreur de reprojection CAM1-Mire : 0.2037 px
[2] EXTRINSÈQUE : Erreur de reprojection CAM2-CAM1 : 0.1461 px
[3] REDRESSEMENT : Erreur géométrique du sol : 0.183 px
[4] TRACKING : Bruit de triangulation YOLO : 4.691 px

-----
📁 SYNTHÈSE DES PRÉCISIONS
-----
🖼 Erreur agrégée totale sur l'image : 5.472 px
🔧 INCERTITUDE DE TRIANGULATION ESTIMÉE À 1.5m : 11.73 mm

```

FIGURE 6.16 – Valeur des erreurs

Sur la figure 6.16, les incertitudes des différentes étapes respectent les seuils que nous nous étions fixés aux sous-section précédentes. Le bruit dû à la triangulation est comme prévue la plus grande source d'erreur. Notre système est précis à $1m50$ à $\sim 1,2cm$, ce qui est très bien.

```
🚀 BILAN DE CORRÉLATION ET PRÉCISION GLOBALE DU SYSTÈME
=====
[1] INTRINSÈQUE : Incertitude de projection CAM1 : 3.1261 px
[1] INTRINSÈQUE : Incertitude de projection CAM2 : 2.4883 px
[1] INTRINSÈQUE : Distorsion CAM1 : 0.0436 px
[1] INTRINSÈQUE : Distorsion CAM2 : 0.0365 px
[2] EXTRINSÈQUE : Erreur de reprojection CAM1-Mire : 0.2037 px
[2] EXTRINSÈQUE : Erreur de reprojection CAM2-CAM1 : 0.1461 px
[3] REDRESSEMENT : Erreur géométrique du sol : 0.183 px
[4] TRACKING : Bruit de triangulation YOLO : 4.691 px

-----
📊 SYNTHÈSE DES PRÉCISIONS
-----
🇫🇷 Erreur agrégée totale sur l'image : 5.472 px
🔧 INCERTITUDE DE TRIANGULATION ESTIMÉE À 3m : 46.92 mm
```

FIGURE 6.17 – Valeur des erreurs

Sur la figure ci-dessus 6.17, on change la distance de détection du BlueROV, les autres données ne changent pas. Pour une distance de mesure à $3m$, la précision est d'environ $4.7cm$ ce qui est très correct.

(Retour au rapport 4.4)